

# SETU

Real-Time Disaster Early-Warning &  
Resource Coordination Platform

“India’s warning system already works.  
What breaks is the six hours *after* the warning.”

A district-scoped coordination platform that closes the loop between citizens, the District Emergency Operations Centre, and rescue forces — ingesting official CAP alerts and geo-tagged citizen reports, deduplicating them into actionable incidents, and solving the resource-allocation problem in real time.

|                |                                                                                                                         |
|----------------|-------------------------------------------------------------------------------------------------------------------------|
| CATEGORY       | Software                                                                                                                |
| THEME          | Disaster Management                                                                                                     |
| DOCUMENT SCOPE | End-to-end architecture, algorithms, API, build plan, evaluation strategy                                               |
| NAME           | SETU — Sanskrit/Hindi for “bridge”. Also: <b>S</b> ituational <b>E</b> valuation & <b>T</b> actical <b>U</b> nification |

# ▪ Contents

---

|           |                                                                                                                                                                           |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>01</b> | Executive Summary                                                                                                                                                         |
| <b>02</b> | Problem Analysis                                                                                                                                                          |
|           | 2.1 The coordination gap · 2.2 Chain of command · 2.3 Failure modes · 2.4 Scope and non-goals                                                                             |
| <b>03</b> | Solution Overview                                                                                                                                                         |
|           | 3.1 The core loop · 3.2 Outcome coverage · 3.3 Personas and journeys                                                                                                      |
| <b>04</b> | System Architecture                                                                                                                                                       |
|           | 4.1 High-level architecture · 4.2 Components · 4.3 Sequence flows · 4.4 Stack decisions · 4.5 Deployment                                                                  |
| <b>05</b> | Data Model                                                                                                                                                                |
|           | 5.1 Entity overview · 5.2 Schema DDL · 5.3 Indexes and partitioning · 5.4 State machines                                                                                  |
| <b>06</b> | The Intelligence Layer — Algorithms                                                                                                                                       |
|           | 6.1 Clustering · 6.2 Severity · 6.3 Trust · 6.4 ETA matrix · 6.5 Optimal assignment · 6.6 Shelter flow · 6.7 Pre-positioning · 6.8 Re-optimization loop · 6.9 Performance |
| <b>07</b> | Backend API Specification                                                                                                                                                 |
| <b>08</b> | Frontend Specification                                                                                                                                                    |
|           | 8.1 Authority dashboard · 8.2 Map layers · 8.3 Citizen PWA and offline design                                                                                             |
| <b>09</b> | Low-Connectivity Channel (SMS / IVR)                                                                                                                                      |
| <b>10</b> | External Data Integration                                                                                                                                                 |
| <b>11</b> | Non-Functional Requirements                                                                                                                                               |
|           | 11.1 Performance · 11.2 Failure modes · 11.3 Security and privacy                                                                                                         |
| <b>12</b> | Repository Structure & Environment                                                                                                                                        |
| <b>13</b> | Build Plan — Phases, Roles, Gates                                                                                                                                         |
| <b>14</b> | Demo Script                                                                                                                                                               |
| <b>15</b> | Evaluation Parameter Mapping                                                                                                                                              |
| <b>16</b> | Q&A Preparation                                                                                                                                                           |
| <b>17</b> | Scalability & Sustainability Model                                                                                                                                        |
| <b>18</b> | Post-Hackathon Roadmap                                                                                                                                                    |
| <b>A</b>  | Appendix — Seed Data Specification                                                                                                                                        |
| <b>B</b>  | Appendix — Environment Variables                                                                                                                                          |
| <b>C</b>  | Appendix — Test & Load Plan                                                                                                                                               |
| <b>D</b>  | Appendix — References & Data Sources                                                                                                                                      |

## HOW TO USE THIS DOCUMENT

Sections 4–11 are the engineering specification — build directly from them. Sections 13–16 are the competition layer: what to build in what order, what to say, and what you will be asked. Section 6 is the part that wins; if time is short, protect it.

# 01 Executive Summary

## 1.1 The one-sentence thesis

India's disaster *warning* pipeline is mature. IMD, CWC, INCOIS and NDMA's SACHET platform push CAP-compliant alerts to citizens by SMS, app, browser and RSS. Mass evacuation works: the 1999 Odisha super cyclone killed roughly 10,000 people; Cyclone Fani in 2019 struck at comparable intensity and killed 89, after 1.2 million people were moved in 48 hours.

And yet floods alone still take about **1,600 lives a year**, affect **7.5 million hectares**, and cause **Rs 1,805 crore** of average annual damage. That residual is not a detection failure. It is a **coordination and allocation failure** in the hours between the alert and the rescue — when a District Emergency Operations Centre is running a live multi-agency operation on WhatsApp groups, phone calls, and a paper register.

### THESIS

**SETU is not another alerting app. It is the missing allocation layer between the alert and the boots on the ground.** It consumes official warnings, opens a return channel from citizens, and converts a chaotic flood of reports into an optimally dispatched response.

## 1.2 What we build

| LAYER    | FUNCTION                                                                                                                                    |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Ingest   | SACHET/IMD CAP alerts, citizen PWA reports (geo-tagged, photo/text), inbound SMS and IVR, field-unit status updates                         |
| Reason   | Spatial clustering of duplicate reports into incidents; severity scoring; trust/corroboration scoring against misinformation                |
| Allocate | Optimal team-incident assignment (Hungarian), shelter overflow routing (min-cost flow), predictive pre-positioning of idle units (p-median) |
| Act      | Live map-based common operating picture for the DEOC, dispatch orders to units, targeted outbound SMS to affected citizens                  |
| Account  | Full audit log of every decision; replayable digital twin of the event; auto-generated after-action report                                  |

## 1.3 The three defensible differentiators

1. **A return channel that does not exist today.** SACHET is broadcast-only — top-down, one-way. There is no upward path from a citizen in the water back to the DEOC, and no allocation layer at all. We consume CAP and add the two missing directions.
2. **Optimization instead of intuition.** Every competing solution dispatches greedily (nearest team first) or manually. We solve the assignment problem to optimality with the Hungarian algorithm, and we show the delta live on stage with a toggle.
3. **Predictive pre-positioning.** The moment a CAP polygon lands, we build an expected-demand surface and solve a facility-location problem to move idle resources *before the first incident exists*. No incumbent does this.

**200→12**

REPORTS COLLAPSED INTO  
INCIDENTS

**34→21**

MEAN RESPONSE,  
MINUTES

**<2s**

REPORT TO DASHBOARD,  
P95

**4**

INGEST CHANNELS,  
DEGRADING

Target figures, measured on the seeded district simulation described in Appendix C.

## 02 Problem Analysis

### 2.1 Where the gap actually is

Disaster response in India decomposes into four phases. Understanding which one is broken is the whole design decision.

| PHASE         | CURRENT NATIONAL CAPABILITY                                                                            | STATE  |
|---------------|--------------------------------------------------------------------------------------------------------|--------|
| Detection     | IMD, CWC (175 flood-forecast stations), INCOIS, GSI — mature, scientific, well-funded                  | Solved |
| Dissemination | NDMA SACHET: CAP-based, geo-targeted, 12 languages, SMS + app + browser + RSS + satellite              | Solved |
| Coordination  | WhatsApp groups, phone calls, paper registers, verbal radio. No shared state, no dedup, no audit trail | BROKEN |
| Allocation    | Human intuition under extreme cognitive load. No optimization, no live capacity view                   | BROKEN |

#### DESIGN CONSEQUENCE

We deliberately build *nothing* in the first two rows. Competing with IMD on forecasting or with NDMA on dissemination is both technically hopeless and strategically wrong. We plug into both and own the two broken rows. This scope discipline is itself an argument for the design.

### 2.2 The chain of command we are building for

Under the Disaster Management Act 2005, response is legally district-led. The **District Collector is the Incident Commander**, operating from the **District Emergency Operations Centre (DEOC)**. That room — not a national control centre — is the user of this product. Every design decision follows from that.

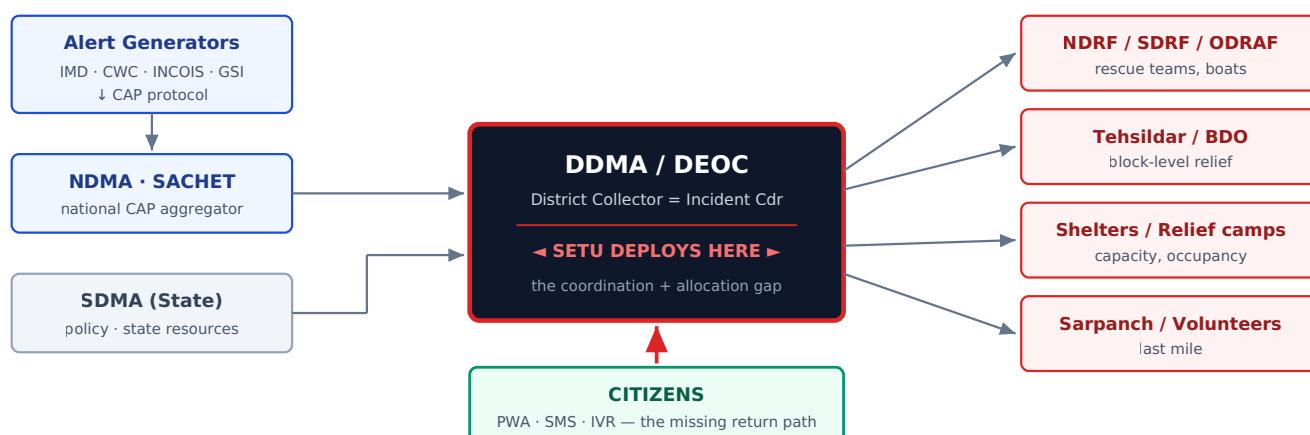


Fig. 2.1 — The national response chain. Red arrow = the return channel that does not exist today.

## 2.3 Failure modes we are attacking

This table is the core of the problem analysis. Each row is an observed operational failure, its root cause, and the specific mechanism in SETU that addresses it.

| OBSERVED FAILURE                                       | ROOT CAUSE                                                                                                | SETU MECHANISM                                                                                  |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Coordination runs on WhatsApp groups and phone calls   | No shared state; nothing queryable, nothing auditable; context dies with the shift change                 | Single map-based common operating picture, persisted and replayable                             |
| 200 calls describing the same collapsed bridge         | No deduplication; operator triages by whoever shouts loudest or calls most                                | Spatio-temporal DBSCAN clustering: reports → incidents (§6.1)                                   |
| Nearest-team dispatch by human judgement               | Cognitively impossible beyond ~15 simultaneous incidents; local decisions compound into globally bad ones | Hungarian optimal assignment over a real road-network cost matrix (§6.5)                        |
| Relief concentrates where cameras and phone signal are | Salience bias; incoming call volume is a proxy for connectivity, not for need                             | Severity score weighted by population and vulnerability, not by report count (§6.2)             |
| Nobody knows which shelter is already full             | Occupancy reported on paper at end of day; evacuees arrive at a full camp and are turned away             | Live capacity ledger + min-cost-flow overflow routing (§6.6)                                    |
| No-internet zones go dark exactly when it matters      | Towers fail or congest; every civic app assumes 4G                                                        | Offline-first PWA queue, inbound SMS grammar, IVR DTMF tree (§9)                                |
| Panic forwards and rumours flood the channel           | No provenance or corroboration model on citizen input                                                     | Trust score on GPS accuracy, media presence, reporter history, independent corroboration (§6.3) |
| Compensation and audit take months after the event     | No machine record of who was rescued, when, by whom, or what was distributed                              | Immutable decision log → auto-generated after-action report (§11.3)                             |

## 2.4 Scope and non-goals

| IN SCOPE                                                                                                                                                                                                                                                             | EXPLICITLY NOT IN SCOPE                                                                                                                                                                                                                                                                                                                                         |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Consuming official CAP alerts<br>Citizen incident reporting across four channels<br>Deduplication, severity triage, trust scoring<br>Resource and shelter allocation optimization<br>DEOC operating picture and dispatch<br>Audit, replay and after-action reporting | Weather forecasting — <i>IMD does this better than we ever could</i><br>Public mass alerting — <i>SACHET already owns this; we consume it</i><br>Hydrological modelling / flood inundation simulation<br>Long-term rehabilitation and compensation disbursal<br>Replacing existing IRS command structure — <i>we mirror it</i><br>Satellite imagery acquisition |

### SAY THIS IN THE PITCH

“We do not forecast weather. We do not replace SACHET. We are the layer *below* the alert and *above* the boots.” Scope discipline reads as engineering maturity, not as limitation.

## 03 Solution Overview

### 3.1 The core loop

Everything in this document exists to serve one loop. If a feature does not appear in this loop or directly support it, it is a P2 item at best.

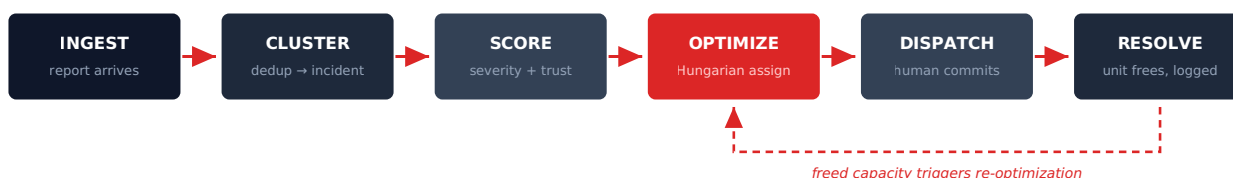


Fig. 3.1 — The core loop. Target: under 2 seconds from ingest to a redrawn dispatch on the DEOC screen.

### 3.2 Coverage of the four required outcomes

The problem statement names four expected outcomes. Judges check these off literally. Each must have a visible on-screen moment in the demo — a brilliant optimizer with a missing SMS path scores worse than four adequate deliverables.

| # | REQUIRED OUTCOME                              | IMPLEMENTATION                                                                                                                            | SECTION  |
|---|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 1 | Citizen reporting app, geo-tagged, photo/text | React PWA: Geolocation API, camera capture, client-side compression, IndexedDB offline queue with background sync                         | §8.3     |
| 2 | Live authority dashboard with heatmaps        | MapLibre GL: severity-weighted heatmap layer, incident pins, resource markers, dispatch lines, 6-hour time slider, live WebSocket updates | §8.1–8.2 |
| 3 | Resource/shelter allocation optimizer         | Hungarian assignment + min-cost-flow shelter routing + p-median pre-positioning, with a live Greedy↔Optimized comparison toggle           | §6       |
| 4 | SMS/IVR fallback for no-internet zones        | Pluggable gateway interface (Twilio adapter + mock adapter), structured SMS grammar parser, IVR DTMF tree, outbound targeted advisories   | §9       |

### 3.3 Personas and journeys

#### PERSONA A — DEOC OPERATIONS OFFICER (PRIMARY USER)

Context: a 3×2 metre video wall, three staff, 400 incoming calls an hour, the Collector asking for a status every ten minutes.

| NEED                   | JOURNEY IN SETU                                                                                                                                                            |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Know what is happening | Opens dashboard → sees CAP alert polygon, clustered incidents ranked by severity, and every unit's live status on one map                                                  |
| Decide what to do      | Sees the recommended assignment set with ETAs and a total-response-time figure; inspects any incident's severity breakdown to understand <i>why</i> it ranks where it does |

|               |                                                                                                                            |
|---------------|----------------------------------------------------------------------------------------------------------------------------|
| Act           | Clicks <b>Commit</b> — unit is notified, status flips to <b>enroute</b> , dispatch line draws on the map, SLA timer starts |
| Override      | Drags an assignment to a different unit; the override and its reason are logged and feed the after-action report           |
| Report upward | One-click situation snapshot: incidents open/closed, people evacuated, shelter occupancy, units committed                  |

## PERSONA B — CITIZEN IN THE AFFECTED ZONE

| NEED                 | JOURNEY IN SETU                                                                                                                         |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Report trouble fast  | Opens PWA (no install, no login) → hazard type, severity 1-5, optional photo, auto GPS → submit in under 20 seconds                     |
| Report with no data  | SMS <b>FL00D 4 761008 water 3ft near school</b> to the district shortcode, or calls the IVR and presses 1                               |
| Know what to do next | Receives a <i>specific</i> outbound instruction — “Shelter: Govt HS Chatrapur, 1.2 km NE, capacity available” — not a generic broadcast |
| Know they were heard | Report shows <b>received → verified → team assigned → resolved</b> with a reference ID                                                  |

## PERSONA C — FIELD RESCUE UNIT (NDRF/ODRAF TEAM LEAD)

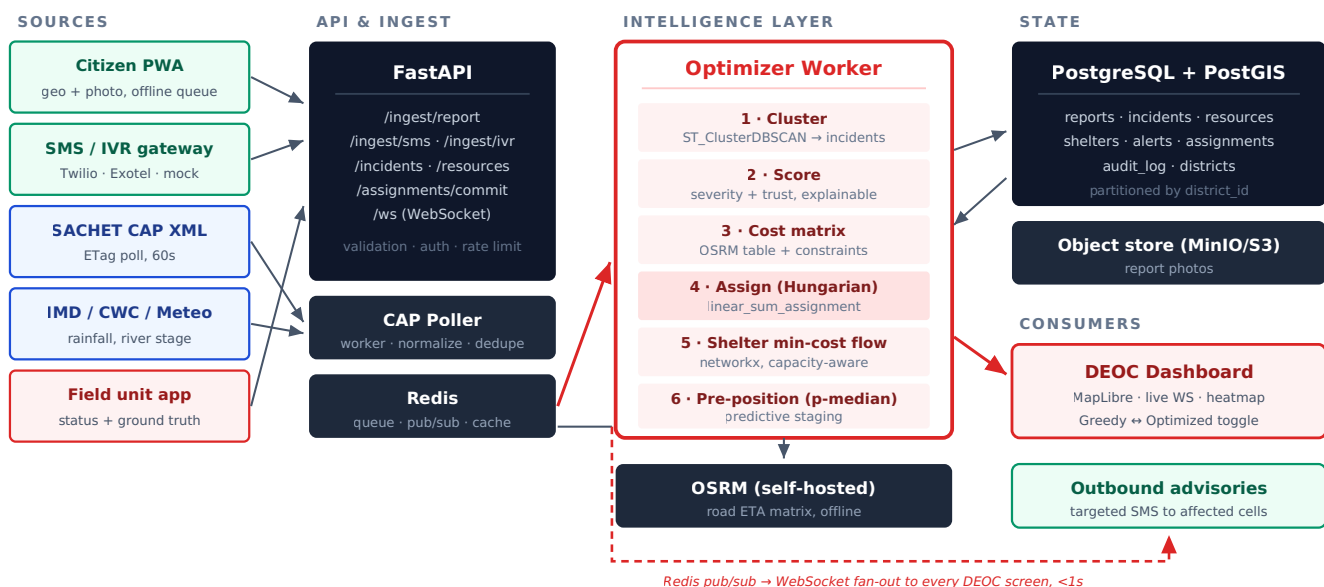
| NEED                         | JOURNEY IN SETU                                                                                                               |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Receive a clear task         | Mobile view: incident location, route, people estimate, hazard type, corroborating photos, contact for the reporter           |
| Update status with one thumb | Three large buttons: <b>Enroute</b> / <b>On site</b> / <b>Resolved</b> . Each flips capacity in the optimizer within a second |
| Report ground truth          | Can escalate severity or add a resource request (“need medical, 12 injured”) which re-enters the loop as a high-trust report  |

## PERSONA D — VOLUNTEER / NGO COORDINATOR (SECONDARY)

Registers a capability (boat, vehicle, 40 food packets) and receives tasking from the same optimizer, tagged as unofficial capacity. This solves the well-documented pattern where NGOs duplicate relief in the same three accessible villages while remote wards receive nothing.

## 04 System Architecture

### 4.1 High-level architecture



#### DEGRADATION LADDER

4G PWA → 2G SMS → IVR voice call → radio operator manual entry into DEOC console. Each rung uses the same ingest contract; the optimizer never knows the difference.

#### MULTI-TENANCY

Every table carries district\_id. One deployment serves N districts; the optimizer runs per-district and is embarrassingly parallel across them. National scale = 766 small independent problems.

Fig. 4.1 — End-to-end system architecture.

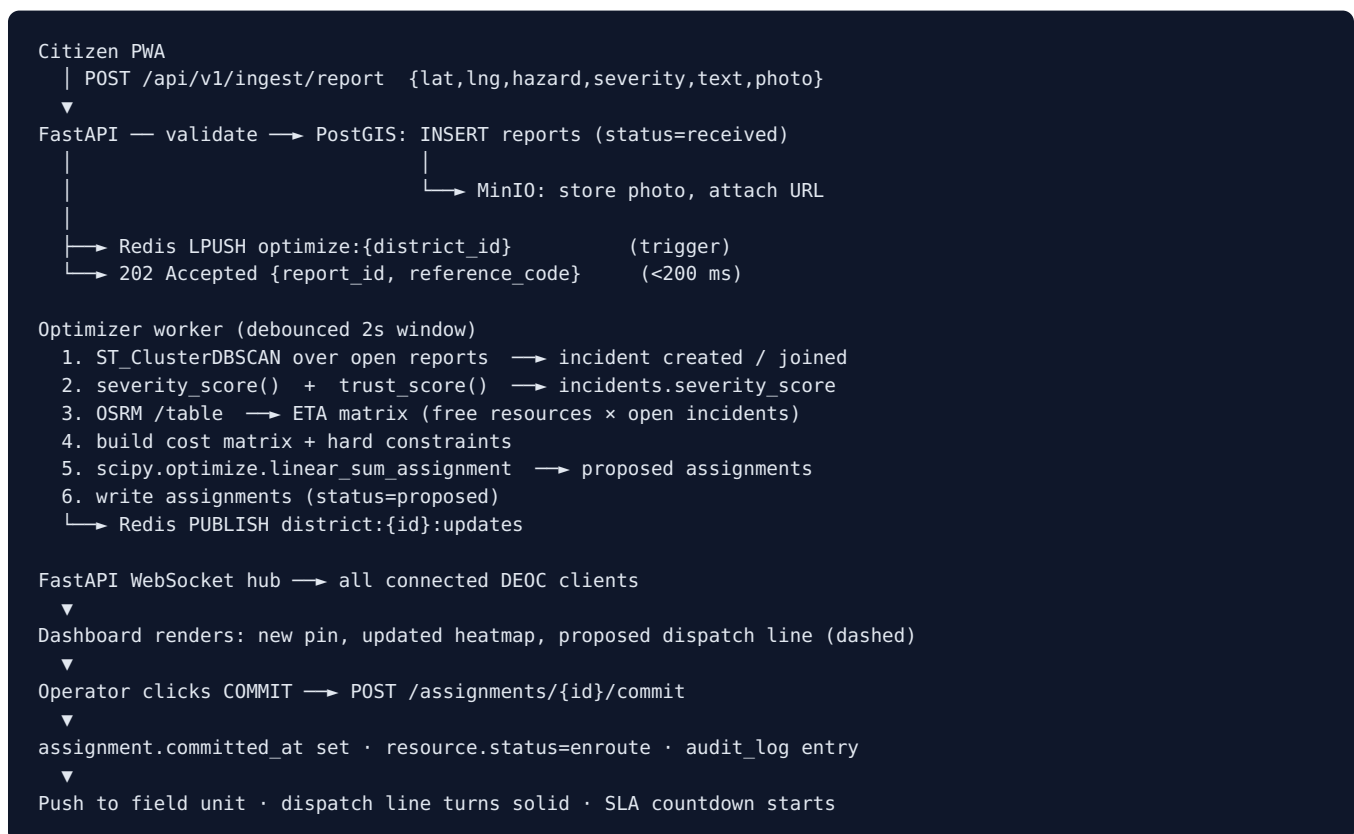
### 4.2 Component responsibilities

| COMPONENT        | RESPONSIBILITY                                                                                                                                | TECHNOLOGY                            |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| Citizen PWA      | Capture geo-tagged report with optional photo; compress client-side; queue offline in IndexedDB; sync on reconnect; show report status        | React + Vite, Workbox                 |
| API gateway      | Validate and authenticate all ingest; rate-limit; write to DB; enqueue an optimization trigger; serve read models; hold WebSocket connections | FastAPI + SQLAlchemy + Pydantic       |
| CAP poller       | Poll SACHET/IMD every 60s with <b>If-None-Match</b> ; parse CAP XML; normalize into <b>alerts</b> ; store area polygon; emit alert event      | Python worker, httpx, lxml            |
| SMS/IVR adapter  | Receive gateway webhooks; parse the SMS grammar or DTMF sequence; geocode pincode/landmark to a point; emit a normalized report               | FastAPI router + adapter interface    |
| Optimizer worker | The whole intelligence layer: cluster → score → cost matrix → assign → shelter flow → pre-position. Runs on event and on a 30s tick           | Python, scipy, networkx, scikit-learn |

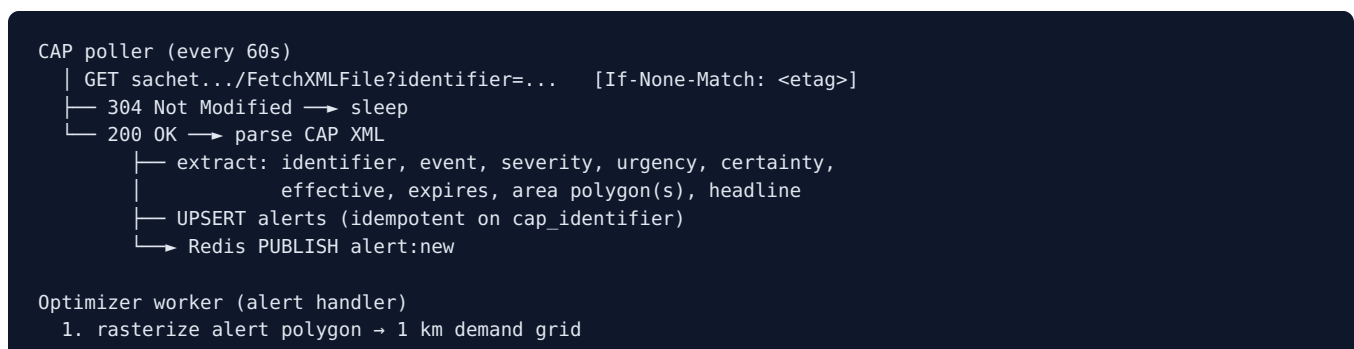
|                       |                                                                                                                    |                               |
|-----------------------|--------------------------------------------------------------------------------------------------------------------|-------------------------------|
| <b>OSRM</b>           | Road-network travel-time matrix between every resource and every incident, computed offline from an OSM extract    | OSRM backend in Docker        |
| <b>PostGIS</b>        | Authoritative state; all spatial operations (clustering, radius, polygon intersection) executed in-database        | PostgreSQL 16 + PostGIS 3.4   |
| <b>Redis</b>          | Optimization job queue, WebSocket pub/sub fan-out, ETag and external-response cache                                | Redis 7                       |
| <b>DEOC dashboard</b> | The common operating picture: map, layers, heatmap, incident queue, resource roster, dispatch, metrics bar, replay | React + MapLibre GL + Zustand |
| <b>Object store</b>   | Report photographs, keyed by report UUID, with signed read URLs                                                    | MinIO (S3-compatible)         |

## 4.3 Sequence flows

### FLOW A — CITIZEN REPORT TO DISPATCHED TEAM



### FLOW B — CAP ALERT TO PRE-POSITIONING RECOMMENDATION



2. weight cells: population × hazard severity × historical incident density × road-access fragility
3. candidate staging points = shelters ∪ police stations ∪ block HQ
4. weighted p-median over candidates → k staging locations
5. assign idle resources → staging points (Hungarian again)
6. write assignments (kind=prepositioning, status=proposed)

▼

Dashboard: amber "PRE-POSITION" banner + recommended moves on map  
Operator commits → units move before the first incident is reported

## FLOW C — SMS REPORT FROM A NO-INTERNET ZONE

Feature phone — "FLOOD 4 761008 water 3ft near school" → gateway shortcode

▼

POST /api/v1/ingest/sms (gateway webhook, HMAC-verified)

▼

parse\_sms() — token 0 = hazard keyword (multi-lingual table)  
— token 1 = severity 1..5  
— token 2 = 6-digit pincode → centroid lookup  
— rest = free text landmark

▼

geocode: pincode centroid, else district centroid; accuracy\_m = 3000

▼

INSERT reports (source='sms', gps\_accuracy\_m=3000, trust adjusted)

▼

identical downstream path to Flow A

▼

outbound SMS reply: "Received. Ref #A7K2. Nearest shelter: Govt HS Chatrapur 1.2km NE. Rescue team assigned."

## 4.4 Technology decisions — with rejected alternatives

Stating what you rejected and why is what separates an engineering document from a feature list. This table is also a pitch slide.

| CHOSEN                      | RATIONALE                                                                                                          | REJECTED                 | WHY NOT                                                                                           |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------|--------------------------|---------------------------------------------------------------------------------------------------|
| <b>PostgreSQL + PostGIS</b> | Clustering, radius search and polygon intersection execute in SQL; no geo code in the app layer; battle-tested     | MongoDB geo              | Weaker spatial operators; no <b>ST_ClusterDBSCAN</b> equivalent; no transactional capacity ledger |
| <b>MapLibre GL JS</b>       | Open source, no token, native heatmap layer, vector tiles, GPU rendering                                           | Mapbox GL / Google Maps  | API key = single point of demo failure; quota and billing risk; licence restrictions              |
| <b>Self-hosted OSRM</b>     | Real road travel times; works with the venue internet disconnected; one <b>/table</b> call returns the full matrix | Google Directions API    | Cost, rate limits, hard network dependency during the demo                                        |
| <b>Hungarian (scipy)</b>    | Provably optimal, $O(n^3)$ , sub-second at district scale, ~10 lines to integrate                                  | OR-Tools VRP solver      | Solves a harder problem than we have; slower to integrate; no additional demo value               |
| <b>React PWA</b>            | Installs by QR in seconds, camera + geolocation + offline queue, one codebase, runs on the judge's own phone       | Flutter / native Android | A full day lost to build tooling and store friction for zero judged value                         |

|                       |                                                                                    |              |                                                                          |
|-----------------------|------------------------------------------------------------------------------------|--------------|--------------------------------------------------------------------------|
| <b>FastAPI</b>        | Async ingest, native WebSockets, Pydantic validation, auto OpenAPI docs            | Django REST  | Heavier, sync-first, no first-class WebSocket story                      |
| <b>Redis pub/sub</b>  | Sub-second fan-out to every DEOC screen; doubles as job queue and ETag cache       | HTTP polling | Latency and load scale badly with screens x incidents                    |
| <b>Docker Compose</b> | Entire stack including OSRM and seed data comes up with one command, fully offline | Kubernetes   | Operational overhead with no benefit at district scale or in a hackathon |

## 4.5 Deployment topology

```
# docker-compose.yml – the entire system, offline-capable
services:
  db:      postgres/postgis:16-3.4      # state
  redis:   redis:7-alpine               # queue + pubsub
  minio:   minio/minio                 # photo object store
  osrm:    osrm/osrm-backend           # pre-built odisha.osrm, road ETAs
  api:     ./backend                   # FastAPI, uvicorn, 2 workers
  worker:  ./backend (celery/arq)       # CAP poller + optimizer
  dashboard: ./frontend-dashboard      # nginx serving built React
  pwa:     ./frontend-pwa              # nginx serving built PWA

# One command, no internet required:
docker compose up --build
# Seeds: district polygon, 50 shelters, 30 resources, OSM extract, replay fixtures
```

### DEMO-DAY RULE

The full stack must come up and run a complete simulation with the venue Wi-Fi *physically disconnected*. Test this by actually turning the Wi-Fi off, at least twice, on the machine that will be used on stage.

## 05 Data Model

### 5.1 Entity overview

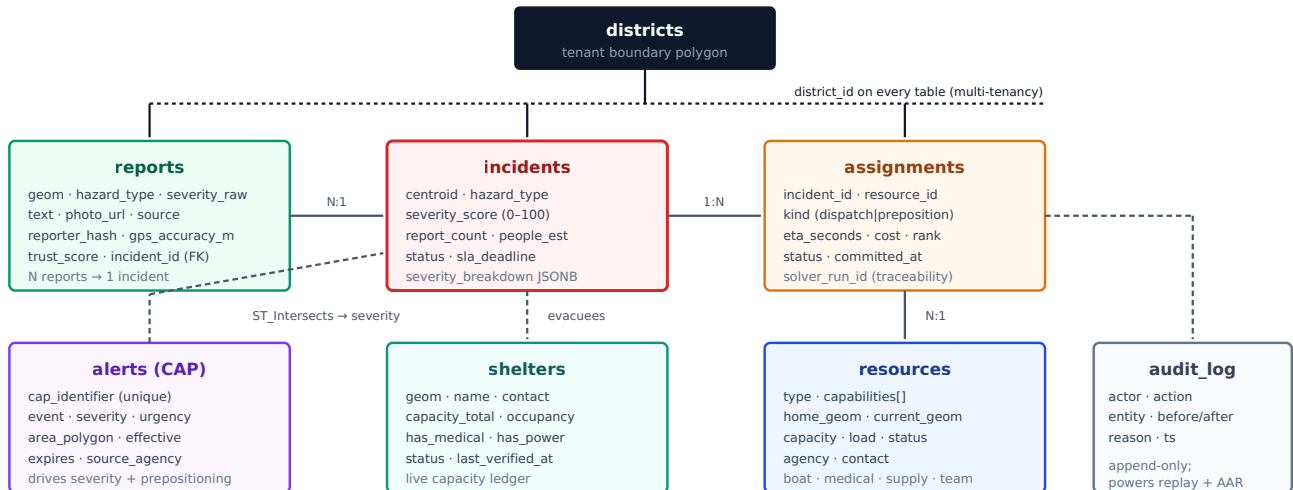


Fig. 5.1 — Entity relationships. Solid = foreign key, dashed = spatial or logical relation.

### 5.2 Schema DDL

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- — tenancy —————
CREATE TABLE districts (
  id          SERIAL PRIMARY KEY,
  name        TEXT NOT NULL,
  state       TEXT NOT NULL,
  lgd_code    TEXT UNIQUE,           -- official LGD district code
  boundary    geography(POLYGON,4326) NOT NULL,
  centroid    geography(POINT,4326) NOT NULL,
  timezone    TEXT DEFAULT 'Asia/Kolkata'
);

-- — official alerts (CAP) —————
CREATE TABLE alerts (
  id          BIGSERIAL PRIMARY KEY,
  district_id INT REFERENCES districts(id),
  cap_identifier TEXT UNIQUE NOT NULL, -- idempotency key
  source_agency TEXT,                 -- IMD | CWC | INCOIS | GSI
  event       TEXT,                   -- "Heavy Rainfall Warning"
  severity    TEXT,                   -- Extreme|Severe|Moderate|Minor
  urgency     TEXT,                   -- Immediate|Expected|Future
  certainty   TEXT,
  headline    TEXT,
  instruction TEXT,
  area_polygon geography(MULTIPOLYGON,4326),
  effective_from TIMESTAMPTZ,
  expires_at   TIMESTAMPTZ,
  raw_xml      TEXT,
  ingested_at  TIMESTAMPTZ DEFAULT now()
);
```

```

-- — citizen reports —————
CREATE TYPE report_source AS ENUM ('app','sms','ivr','field_unit','manual');
CREATE TYPE hazard AS ENUM ('flood','landslide','cyclone_damage','building_collapse',
                             'medical','fire','stranded','power_line','other');

CREATE TABLE reports (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  district_id       INT NOT NULL REFERENCES districts(id),
  geom              geography(POINT,4326) NOT NULL,
  gps_accuracy_m    INT,                -- 5 for GPS, 3000 for pincode
  hazard_type        hazard NOT NULL,
  severity_raw       SMALLINT CHECK (severity_raw BETWEEN 1 AND 5),
  description        TEXT,
  photo_url         TEXT,
  source            report_source NOT NULL,
  reporter_hash     TEXT,                -- HMAC(phone, salt) — never raw PII
  people_reported   INT,
  trust_score        NUMERIC(4,3) DEFAULT 0.5,
  incident_id       BIGINT REFERENCES incidents(id),
  status            TEXT DEFAULT 'received',
  reference_code     TEXT UNIQUE,        -- short code SMS'd back
  created_at        TIMESTAMPTZ DEFAULT now()
);

-- — incidents (clustered reports) —————
CREATE TABLE incidents (
  id                BIGSERIAL PRIMARY KEY,
  district_id       INT NOT NULL REFERENCES districts(id),
  centroid          geography(POINT,4326) NOT NULL,
  hazard_type        hazard NOT NULL,
  severity_score     NUMERIC(5,2) DEFAULT 0,    -- 0..100
  severity_breakdown JSONB,                  -- explainability payload
  report_count       INT DEFAULT 1,
  people_affected_est INT,
  status            TEXT DEFAULT 'open',    -- open|assigned|onsite|resolved|false_alarm
  sla_deadline       TIMESTAMPTZ,
  opened_at         TIMESTAMPTZ DEFAULT now(),
  resolved_at       TIMESTAMPTZ
);

-- — resources —————
CREATE TYPE resource_type AS ENUM ('rescue_team','boat','ambulance','medical_team',
                                    'supply_truck','heavy_equipment','volunteer_group');

CREATE TABLE resources (
  id                BIGSERIAL PRIMARY KEY,
  district_id       INT NOT NULL REFERENCES districts(id),
  name              TEXT NOT NULL,
  type              resource_type NOT NULL,
  agency            TEXT,                  -- NDRF | ODRAF | Fire | NGO
  capabilities       TEXT[] DEFAULT '{}',  -- {water_rescue,medical,cutting}
  home_geom         geography(POINT,4326) NOT NULL,
  current_geom      geography(POINT,4326) NOT NULL,
  capacity          INT DEFAULT 1,        -- people rescuable / units of supply
  load              INT DEFAULT 0,
  status            TEXT DEFAULT 'idle',   -- idle|enroute|onsite|returning|offline
  contact           TEXT,
  last_ping_at      TIMESTAMPTZ DEFAULT now()
);

-- — shelters —————
CREATE TABLE shelters (
  id                BIGSERIAL PRIMARY KEY,
  district_id       INT NOT NULL REFERENCES districts(id),
  name              TEXT NOT NULL,
  geom              geography(POINT,4326) NOT NULL,
  capacity_total    INT NOT NULL,
  occupancy         INT DEFAULT 0,
  has_medical       BOOLEAN DEFAULT false,
  has_power         BOOLEAN DEFAULT false,

```

```

has_water      BOOLEAN DEFAULT false,
status         TEXT DEFAULT 'open', -- open|full|closed|inaccessible
contact        TEXT,
last_verified_at TIMESTAMPTZ
);

-- — assignments (solver output) —————
CREATE TABLE assignments (
  id            BIGSERIAL PRIMARY KEY,
  district_id  INT NOT NULL REFERENCES districts(id),
  incident_id   BIGINT REFERENCES incidents(id),
  resource_id   BIGINT REFERENCES resources(id),
  shelter_id    BIGINT REFERENCES shelters(id),
  kind          TEXT NOT NULL,        -- dispatch | preposition | evacuation
  eta_seconds   INT,
  cost          NUMERIC(10,3),
  solver_run_id UUID,                -- traceability to one solver invocation
  strategy      TEXT,                -- greedy | optimized (for the toggle)
  status        TEXT DEFAULT 'proposed', -- proposed|committed|completed|cancelled
  created_at    TIMESTAMPTZ DEFAULT now(),
  committed_at  TIMESTAMPTZ,
  completed_at  TIMESTAMPTZ
);

-- — audit (append-only) —————
CREATE TABLE audit_log (
  id            BIGSERIAL PRIMARY KEY,
  district_id  INT NOT NULL,
  actor        TEXT NOT NULL,        -- user id or 'system:optimizer'
  action       TEXT NOT NULL,        -- commit_assignment | override | resolve
  entity_type  TEXT, entity_id TEXT,
  before       JSONB, after JSONB,
  reason       TEXT,
  ts           TIMESTAMPTZ DEFAULT now()
);

```

## 5.3 Indexes and partitioning

```

-- spatial indexes: mandatory, everything below depends on them
CREATE INDEX idx_reports_geom      ON reports USING GIST (geom);
CREATE INDEX idx_incidents_geom    ON incidents USING GIST (centroid);
CREATE INDEX idx_resources_geom    ON resources USING GIST (current_geom);
CREATE INDEX idx_shelters_geom     ON shelters USING GIST (geom);
CREATE INDEX idx_alerts_area       ON alerts USING GIST (area_polygon);

-- hot query paths
CREATE INDEX idx_reports_open      ON reports (district_id, created_at DESC)
  WHERE incident_id IS NULL;
CREATE INDEX idx_incidents_active  ON incidents (district_id, severity_score DESC)
  WHERE status IN ('open','assigned');
CREATE INDEX idx_resources_free    ON resources (district_id, type)
  WHERE status = 'idle';
CREATE INDEX idx_alerts_live       ON alerts (district_id, expires_at)
  WHERE expires_at > now();

-- scale path (not needed for the hackathon; state it in the pitch)
-- reports & audit_log PARTITION BY RANGE (created_at) with monthly partitions,
-- sub-partitioned by district_id hash for multi-district deployments.

```

## 5.4 State machines

Explicit state machines prevent the classic demo bug where a unit is dispatched twice or an incident closes while a team is still en route.

| ENTITY            | TRANSITIONS                                                                                                                                                                                                                                                   |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>report</b>     | <code>received</code> → <code>clustered</code> → <code>verified</code> → <code>resolved</code>   <code>received</code> → <code>quarantined</code> (trust below threshold) → <code>discarded</code> / <code>verified</code>                                    |
| <b>incident</b>   | <code>open</code> → <code>assigned</code> → <code>onsite</code> → <code>resolved</code>   <code>open</code> → <code>false_alarm</code> . Re-entry to <code>open</code> allowed only on explicit operator reopen (logged)                                      |
| <b>resource</b>   | <code>idle</code> → <code>enroute</code> → <code>onsite</code> → <code>returning</code> → <code>idle</code> . <code>offline</code> reachable from any state (comms loss). Only <code>idle</code> and <code>returning</code> enter the solver's candidate pool |
| <b>assignment</b> | <code>proposed</code> → <code>committed</code> → <code>completed</code> . <code>proposed</code> → <code>cancelled</code> on re-optimization. <b>Committed assignments are immutable inputs to the next solver run</b> — this is the anti-thrash guarantee     |
| <b>shelter</b>    | <code>open</code> → <code>full</code> (occupancy ≥ capacity) → <code>open</code> . <code>inaccessible</code> set by field unit; removed from the flow network immediately                                                                                     |

#### INVARIANT TO ENFORCE IN CODE

A resource in `enroute` or `onsite` can never appear in the solver's free pool. Enforce this in SQL ( `WHERE status = 'idle'` ) and assert it again in the worker. Every live-demo dispatch bug traces back to violating this.

## 06 The Intelligence Layer

### THIS IS THE SECTION THAT WINS

Every competing team will demo a map with pins. Almost none will demo a solver. Spend roughly 40% of engineering time here and protect it from scope creep. If something has to be cut, cut from \$8, never from \$6.

### 6.1 Stage 1 — Report clustering (deduplication)

Ten people report the same collapsed bridge. Left raw, the DEOC sees ten tasks and dispatches against noise. Spatio-temporal DBSCAN collapses them into one incident. PostGIS does this natively, so there is no clustering code in the application layer at all.

```
-- Cluster unassigned reports within a 30-minute window.
-- eps = 300 m, minpoints = 1 (a single report is still a valid incident).
WITH recent AS (
  SELECT id, hazard_type, geom::geometry AS g, severity_raw, trust_score
  FROM reports
  WHERE district_id = :did
     AND incident_id IS NULL
     AND trust_score >= 0.35
     AND created_at > now() - interval '30 minutes'
),
clustered AS (
  SELECT id, hazard_type, g, severity_raw, trust_score,
         ST_ClusterDBSCAN(g, eps := 300, minpoints := 1)
         OVER (PARTITION BY hazard_type) AS cid
  FROM recent
)
SELECT hazard_type, cid,
       ST_Centroid(ST_Collect(g))      AS centroid,
       COUNT(*)                      AS report_count,
       AVG(severity_raw)              AS mean_severity,
       AVG(trust_score)               AS mean_trust,
       ARRAY_AGG(id)                 AS report_ids
FROM clustered
GROUP BY hazard_type, cid;
```

#### Design notes

- **Partition by hazard type.** A medical emergency and a flood report 50 m apart are two different incidents requiring two different capabilities. Clustering across hazard types would merge them and dispatch the wrong unit.
- **eps = 300 m** matches typical urban-ward granularity. Make it configurable per district — a dense town needs 150 m, a sparse block needs 800 m.
- **Rolling window.** A report from four hours ago describes a different situation than one from four minutes ago. Reports outside the window attach to closed incidents or open new ones.
- **Merge into existing incidents:** before creating a new incident, check `ST_DWithin(existing.centroid, new_centroid, 300)` with matching hazard type and `status = 'open'`; if found, attach and recompute rather than fragmenting.

#### DEMO LINE

“Two hundred reports arrived in ninety seconds. A human operator cannot triage that. Our system collapsed them into twelve actionable incidents — and it did it in the database, in eleven milliseconds.”

## 6.2 Stage 2 — Severity scoring

The score must be **explainable**. A black box that outputs 87 is useless to a Collector who has to justify a decision. Store the component breakdown in `severity_breakdown` and render it in the incident panel.

```
WEIGHTS = {
    "reported": 0.35, # what people actually said
    "corroboration": 0.20, # independent confirmation
    "hazard": 0.15, # intrinsic lethality of the hazard
    "population": 0.15, # who is exposed
    "official": 0.15, # inside an active CAP alert polygon
}

HAZARD_WEIGHT = {
    # 0.1, from NDMA fatality profiles
    "building_collapse": 1.00, "landslide": 0.95, "medical": 0.90,
    "stranded": 0.80, "flood": 0.70, "fire": 0.85,
    "power_line": 0.60, "cyclone_damage": 0.55, "other": 0.40,
}

def severity_score(inc) -> tuple[float, dict]:
    # 1. mean reported severity, trust-weighted
    reported = (inc.mean_severity / 5.0) * inc.mean_trust

    # 2. corroboration – log-scaled and capped so a brigade of
    # forwarded messages cannot dominate a genuine single report
    corroboration = min(math.log1p(inc.report_count) / math.log(11), 1.0)

    # 3. intrinsic hazard lethality
    hazard = HAZARD_WEIGHT[inc.hazard_type]

    # 4. exposed population, normalised against district max
    population = min(inc.pop_density_1km / DISTRICT_P95_DENSITY, 1.0)

    # 5. official corroboration: inside a live CAP polygon?
    official = CAP_SEVERITY.get(inc.active_alert_severity, 0.0)
    # Extreme 1.0 | Severe 0.75 | Moderate 0.45 | Minor 0.2 | none 0

    parts = {"reported": reported, "corroboration": corroboration,
            "hazard": hazard, "population": population, "official": official}
    score = 100.0 * sum(WEIGHTS[k] * v for k, v in parts.items())

    # hard escalations – override the weighted score
    if inc.hazard_type in ("medical", "building_collapse") and inc.people_reported:
        score = max(score, 85.0)
    if inc.age_minutes > 45 and inc.status == "open":
        score = min(score + 0.25 * (inc.age_minutes - 45), 100.0) # ageing

    return round(score, 2), parts
```

### Why each term exists (be ready to defend all five)

| TERM                 | JUSTIFICATION                                                                                                                                                                        |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>reported</b>      | The person on the ground has information no model has. Trust-weighted so unverifiable input counts less, not zero.                                                                   |
| <b>corroboration</b> | Independent confirmation is genuine evidence. Log-scaled and capped: this is the single most important defence against a coordinated or panicked report flood dominating triage.     |
| <b>hazard</b>        | A building collapse with people inside is not equivalent to ankle-deep waterlogging, whatever the reporters selected.                                                                |
| <b>population</b>    | <b>The equity term.</b> Corrects for the fact that report volume tracks phone density, not need. This is what stops relief concentrating in accessible, connected, photogenic wards. |

|                 |                                                                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>official</b> | Fuses citizen input with IMD/CWC ground truth. A report inside an Extreme CAP polygon is prior-boosted, not treated as an isolated event. |
| <b>ageing</b>   | Prevents starvation: a low-severity incident cannot sit unserved forever while new high-severity ones keep pre-empting it.                |

## 6.3 Stage 3 — Trust and corroboration scoring

Misinformation and panic-forwarding are documented disaster problems. Reports are never auto-rejected — low-trust reports are quarantined into a separate operator queue and excluded from automatic dispatch.

```
def trust_score(r, neighbours) -> float:
    s = 0.5                                # neutral prior
    # provenance
    if r.gps_accuracy_m and r.gps_accuracy_m <= 50: s += 0.15
    elif r.gps_accuracy_m and r.gps_accuracy_m > 1000: s -= 0.10 # pincode-only
    if r.photo_url: s += 0.15
    if r.photo_exif_timestamp_within(minutes=30): s += 0.10
    if r.source == "field_unit": s = 1.00 # authoritative

    # independent corroboration within 300 m / 20 min, distinct reporters
    distinct = len({n.reporter_hash for n in neighbours} - {r.reporter_hash})
    s += min(0.05 * distinct, 0.20)

    # reporter history
    if r.reporter_false_alarms >= 2: s -= 0.25
    if r.reporter_confirmed_reports >= 3: s += 0.10

    # rate-limit signature: same hash, many reports, tiny time window
    if r.reporter_reports_last_10min > 5: s -= 0.30

    return max(0.0, min(1.0, s))

# Policy: < 0.35 → quarantine queue (visible, never auto-dispatched)
#         ≥ 0.35 → enters clustering and can trigger dispatch
#         ≥ 0.80 → eligible for auto-escalation to critical
```

## 6.4 Stage 4 — Travel-time cost matrix

Straight-line distance is wrong precisely when it matters: during a flood the road network is what constrains movement, and a 2 km straight line can be a 25 km detour around a submerged causeway. One OSRM [table](#) call returns the full matrix.

```
async def eta_matrix(resources, incidents) -> np.ndarray:
    """Return [len(resources) x len(incidents)] travel times in seconds."""
    coords = ";;".join(
        [f"{r.lng},{r.lat}" for r in resources] +
        [f"{i.lng},{i.lat}" for i in incidents]
    )
    n_r, n_i = len(resources), len(incidents)
    url = (f"{OSRM}/table/v1/driving/{coords}"
          f"?sources={';'}.join(map(str, range(n_r)))"
          f"&destinations={';'}.join(map(str, range(n_r, n_r + n_i)))")
    data = await http.get(url, timeout=5.0)
    M = np.array(data["durations"], dtype=float) # seconds

    # graceful degradation — OSRM down must not stop dispatch
    if np.isnan(M).any():
        M = np.where(np.isnan(M), haversine_matrix(resources, incidents) / 8.33, M)
        # 8.33 m/s ≈ 30 km/h effective disaster-conditions road speed
```

```
# flood-aware penalty: edges inside an active inundation polygon
# get a multiplier; blocked routes get inf
return apply_route_penalties(M, active_alert_polygons)
```

## PREPARATION WARNING

Download and pre-process the OSM extract for *one state only* (e.g. [odisha-latest.osm.pbf](#), roughly 150 MB) well before the event. [osrm-extract](#) and [osrm-contract](#) on an India-wide extract will take hours and can exhaust RAM. Commit the prepared [.osrm](#) files or a Docker volume.

## 6.5 Stage 5 — Optimal assignment (the differentiator)

This is the algorithmic core. Greedy nearest-first assignment is what every other team will build — it is locally sensible and globally poor, because early greedy choices strand the last incident with a 90-minute drive. The Hungarian algorithm minimises *total* weighted response time and runs in  $O(n^3)$ , which is well under a second at district scale.

```
from scipy.optimize import linear_sum_assignment

BIG = 1e6 # soft-infinity for hard constraint violations

def build_cost_matrix(resources, incidents, eta):
    """cost[i][j] = cost of sending resource i to incident j."""
    C = eta / 60.0 # minutes — the base cost

    for i, r in enumerate(resources):
        for j, inc in enumerate(incidents):
            # — hard constraints —
            if not required_caps(inc).issubset(set(r.capabilities)):
                C[i][j] += BIG # wrong capability
            if inc.people_affected_est > r.capacity:
                C[i][j] += BIG # insufficient capacity
            if r.status not in ("idle", "returning"):
                C[i][j] += BIG # not available
            if inc.status == "resolved":
                C[i][j] += BIG

            # — soft preferences —
            # triage: severe incidents pull resources toward them
            C[i][j] -= inc.severity_score * URGENCY_WEIGHT # 0.30

            # SLA breach risk dominates raw distance
            if inc.sla_deadline:
                slack = (inc.sla_deadline - now()).total_seconds() / 60.0
                if slack < eta[i][j] / 60.0:
                    C[i][j] += SLA_BREACH_PENALTY # 500

            # anti-thrash: do not yank a unit off a committed job
            if r.committed_incident_id == inc.id:
                C[i][j] -= COMMITMENT_BONUS # 200
            elif r.committed_incident_id is not None:
                C[i][j] += REASSIGNMENT_PENALTY # 400

            # agency preference: keep NDRF for the hardest jobs
            if r.agency == "NDRF" and inc.severity_score < 40:
                C[i][j] += 50

    return C

def solve(resources, incidents, eta, strategy="optimized"):
    C = build_cost_matrix(resources, incidents, eta)
```

```

if strategy == "greedy": # baseline, for the live comparison
    assigns, used = [], set()
    for j in sorted(range(len(incidents)),
                    key=lambda j: -incidents[j].severity_score):
        cand = [i for i in range(len(resources))
                if i not in used and C[i][j] < BIG]
        if not cand:
            continue
        i = min(cand, key=lambda i: eta[i][j]) # nearest free unit
        assigns.append((i, j)); used.add(i)
    return assigns

# pad to square – scipy handles rectangular, but padding makes
# the "unassigned" case explicit and auditable
rows, cols = linear_sum_assignment(C)
return [(int(i), int(j)) for i, j in zip(rows, cols) if C[i][j] < BIG]

```

## WHY THIS IS GENUINELY BETTER — THE ARGUMENT TO MAKE ON STAGE

| SCENARIO                                                                                                                                     | GREEDY RESULT                                                                                                                          | HUNGARIAN RESULT                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 boats, 2 incidents. Boat A is 5 min from Incident 1 and 8 min from Incident 2. Boat B is 6 min from Incident 1 but 45 min from Incident 2. | Incident 1 (higher severity) is served first and grabs Boat A (5 min). Boat B then drives 45 min to Incident 2. <b>Total = 50 min.</b> | Boat B → Incident 1 (6 min), Boat A → Incident 2 (8 min). <b>Total = 14 min.</b> One minute worse on the first incident; 37 minutes better overall. |

This two-line example is worth rehearsing verbatim. It makes an abstract algorithm concrete in fifteen seconds, and it is the single clearest demonstration of engineering depth you have.

### BUILD THE TOGGLE

A **Greedy ↔ Optimized** switch in the dashboard header that re-solves live and updates a metric strip: **mean response 34 min → 21 min (-38%), worst-case 91 → 34 min, unassigned critical 3 → 0**. This single control demonstrates novelty, technical depth, and measurable impact simultaneously — three evaluation parameters from one component.

## 6.6 Stage 6 — Shelter allocation as min-cost flow

Assignment handles one-to-one matching. Evacuation is different: many people from one incident may need to split across several shelters because the nearest one fills. That is a capacitated transportation problem, solved exactly by min-cost flow.

```

import networkx as nx

def allocate_shelters(incidents, shelters, eta_is):
    G = nx.DiGraph()
    total = sum(i.people_affected_est for i in incidents)

    # source → incident, capacity = people needing evacuation
    for k, inc in enumerate(incidents):
        G.add_edge("SRC", f"i{k}", capacity=inc.people_affected_est, weight=0)

    for s, sh in enumerate(shelters):
        if sh.status != "open":
            continue
        free = sh.capacity_total - sh.occupancy

```

```

        if free <= 0:
            continue
        # cost = travel minutes, + penalty if the incident needs
        # medical support and this shelter has none
        w = int(eta_is[k][s] / 60)
        if inc.needs_medical and not sh.has_medical:
            w += 30
        G.add_edge(f"i{k}", f"s{s}", capacity=free, weight=w)

# shelter → sink, capacity = remaining beds
for s, sh in enumerate(shelters):
    free = sh.capacity_total - sh.occupancy
    if sh.status == "open" and free > 0:
        G.add_edge(f"s{s}", "SNK", capacity=free, weight=0)

G.nodes["SRC"]["demand"] = -total
G.nodes["SNK"]["demand"] = total

flow = nx.max_flow_min_cost(G, "SRC", "SNK")

plan, unplaced = [], total
for k, inc in enumerate(incidents):
    for s, sh in enumerate(shelters):
        n = flow.get(f"i{k}", {}).get(f"s{s}", 0)
        if n > 0:
            plan.append((inc.id, sh.id, n, eta_is[k][s]))
            unplaced -= n
return plan, unplaced # unplaced > 0 ⇒ district capacity shortfall alarm

```

### Why judges like this

- **It degrades gracefully.** Nearest shelter full → overflow routes to second-nearest automatically. No special-case code.
- **It surfaces a real operational number.** If `unplaced > 0`, the district is short of shelter capacity *right now* — raise an escalation to the SDMA for neighbouring-district support. That is a decision no existing system supports.
- **It handles the medical constraint** without a separate rule engine, just an edge weight.

## 6.7 Stage 7 — Predictive pre-positioning (the novelty claim)

Everything above is reactive. This is not. When a CAP alert lands, we compute where demand is *likely* to appear and move idle resources there before any incident exists.

```

def prepositioning_plan(alert, resources, candidates, k=5):
    """
    alert      : active CAP alert with area_polygon
    candidates : possible staging points (shelters, police stations, block HQs)
    Returns    : recommended moves for currently idle resources.
    """
    # 1. rasterize the alert polygon into a ~1 km demand grid
    cells = rasterize(alert.area_polygon, cell_km=1.0)

    # 2. weight each cell by expected demand
    for c in cells:
        c.w = (
            0.40 * norm(c.population)           # who is exposed
            + 0.25 * norm(c.historical_incident_density) # where it happened before
            + 0.20 * CAP_SEVERITY[alert.severity]      # how bad the warning is
            + 0.15 * norm(c.road_fragility)           # low-lying / single-access
        )

    # 3. weighted p-median: choose k staging points minimising
    # Σ (cell weight × travel time to its nearest staging point)
    best, best_cost = None, float("inf")

```

```

for combo in top_k_combinations(candidates, k, beam=200):
    cost = sum(c.w * min(travel_time(c, s) for s in combo) for c in cells)
    if cost < best_cost:
        best, best_cost = combo, cost

# 4. assign idle resources to staging points – Hungarian again
idle = [r for r in resources if r.status == "idle"]
eta = eta_matrix_sync(idle, best)
rows, cols = linear_sum_assignment(eta)

return [{"resource_id": idle[i].id,
        "stage_at": best[j],
        "eta_min": round(eta[i][j] / 60),
        "expected_demand_covered": coverage(best[j], cells)}
        for i, j in zip(rows, cols)]

```

### THE LINE THAT GETS REPEATED BETWEEN JUDGES

“Every other system reacts to the first call. Ours has already moved the boats — forty minutes before the first call comes in.”

**Effort estimate:** 4–5 hours, because it reuses the ETA matrix and the Hungarian solver you already built. A beam-limited combination search over 30–60 candidate points is entirely adequate; do not reach for a MILP solver. If time is very short, replace the p-median with weighted k-means over demand cells and snap each centroid to the nearest candidate point — visually and narratively identical, one hour of work.

## 6.8 The re-optimization loop

```

async def optimization_cycle(district_id):
    """Runs on: new report · unit status change · alert · 30s tick."""
    async with lock(f"opt:{district_id}"): # one solver at a time
        run_id = uuid4()

        incidents = await cluster_and_score(district_id) # §6.1 §6.2
        free = await fetch_available_resources(district_id)
        if not incidents or not free:
            return

        # pre-filter: 50 km radius keeps the matrix small and OSRM fast
        pairs = spatial_prefilter(free, incidents, radius_km=50)

        eta = await eta_matrix(pairs.resources, pairs.incidents) # §6.4
        opt = solve(pairs.resources, pairs.incidents, eta, "optimized") # §6.5
        grd = solve(pairs.resources, pairs.incidents, eta, "greedy") # baseline

        await persist_assignments(opt, run_id, strategy="optimized")
        await persist_metrics(compare(opt, grd, eta)) # drives the toggle

        shelter_plan, shortfall = allocate_shelters(...) # §6.6
        await persist_shelter_plan(shelter_plan, run_id)
        if shortfall > 0:
            await raise_alarm("SHELTER_CAPACITY_SHORTFALL", shortfall)

        await redis.publish(f"district:{district_id}:updates",
                           json.dumps({"type": "reoptimized", "run_id": str(run_id)}))

```

### Three rules that keep the loop stable

1. **Debounce.** A 2-second collection window; a burst of 80 reports triggers one solver run, not eighty.
2. **Commitment locking.** Once an operator commits an assignment and the unit is **enroute**, that pairing is a fixed input to every subsequent run. Without this the map thrashes and the demo looks broken.

3. **Distributed lock per district.** Two concurrent solver runs on the same district produce conflicting writes. One lock, held for the run.

## 6.9 Complexity and measured performance

| STAGE                       | COMPLEXITY      | SCALE (DISTRICT)           | TARGET LATENCY |
|-----------------------------|-----------------|----------------------------|----------------|
| DBSCAN clustering (PostGIS) | $O(n \log n)$   | 500 open reports           | < 50 ms        |
| Severity + trust scoring    | $O(n)$          | 50 incidents               | < 10 ms        |
| OSRM table request          | $O(r \times i)$ | 60 × 50                    | < 400 ms       |
| Hungarian assignment        | $O(n^3)$        | 60 × 50                    | < 80 ms        |
| Min-cost flow (shelters)    | $O(V^2E)$       | 50 incidents × 60 shelters | < 250 ms       |
| p-median pre-positioning    | beam-limited    | 60 candidates, k = 5       | < 900 ms       |
| Full cycle                  |                 |                            | < 1.5 s        |

Above roughly 500 incidents in one district the  $O(n^3)$  term begins to matter. The mitigation is already in the loop: the 50 km spatial pre-filter keeps the effective matrix small, and beyond that the district itself is the natural shard. State this when asked about scale — it shows you know where your own design breaks.

## 07 Backend API Specification

### 7.1 REST endpoints

| METH<br>OD | PATH                                           | PURPOSE                                                                                            | AUTH                |
|------------|------------------------------------------------|----------------------------------------------------------------------------------------------------|---------------------|
| POST       | <code>/api/v1/ingest/report</code>             | Citizen report from the PWA (multipart with optional photo)                                        | public + rate limit |
| POST       | <code>/api/v1/ingest/sms</code>                | Gateway webhook, HMAC-verified; parses the SMS grammar                                             | HMAC                |
| POST       | <code>/api/v1/ingest/ivr</code>                | IVR DTMF callback; builds a report from key presses                                                | HMAC                |
| GET        | <code>/api/v1/reports/{ref}</code>             | Citizen-facing status of one report by reference code                                              | public              |
| GET        | <code>/api/v1/incidents</code>                 | List incidents; filters: status, hazard, min_severity, bbox                                        | operator            |
| GET        | <code>/api/v1/incidents/{id}</code>            | Detail: reports, photos, severity breakdown, assignment history                                    | operator            |
| PATCH      | <code>/api/v1/incidents/{id}</code>            | Operator override: status, severity, people estimate (audited)                                     | operator            |
| GET        | <code>/api/v1/resources</code>                 | Roster with live status and location                                                               | operator            |
| PATCH      | <code>/api/v1/resources/{id}/status</code>     | Field-unit status update: enroute / onsite / resolved                                              | field               |
| GET        | <code>/api/v1/shelters</code>                  | Shelters with live occupancy and facilities                                                        | operator            |
| PATCH      | <code>/api/v1/shelters/{id}</code>             | Update occupancy or mark inaccessible                                                              | operator            |
| GET        | <code>/api/v1/assignments?strategy=</code>     | Current plan; <code>strategy=greedy optimized</code> powers the toggle                             | operator            |
| POST       | <code>/api/v1/assignments/{id}/commit</code>   | Human commits a proposed dispatch                                                                  | operator            |
| POST       | <code>/api/v1/assignments/{id}/override</code> | Reassign to a different unit with a logged reason                                                  | operator            |
| POST       | <code>/api/v1/optimize</code>                  | Force a solver run (also used by the demo simulator)                                               | operator            |
| GET        | <code>/api/v1/alerts/active</code>             | Live CAP alerts with area polygons                                                                 | public              |
| GET        | <code>/api/v1/metrics</code>                   | Dashboard strip: open incidents, mean response, units free, shelter occupancy, unassigned critical | operator            |
| POST       | <code>/api/v1/simulate/{scenario}</code>       | Spawn a scripted event stream — the demo backbone                                                  | operator            |
| GET        | <code>/api/v1/replay?from=&amp;to=</code>      | Time-ordered event stream for digital-twin replay                                                  | operator            |
| GET        | <code>/api/v1/aar/{event_id}.pdf</code>        | Generated after-action report                                                                      | operator            |

### 7.2 Representative contracts

```
// POST /api/v1/ingest/report
{
  "lat": 19.3149, "lng": 84.7941, "gps_accuracy_m": 12,
  "hazard_type": "flood", "severity_raw": 4,
  "description": "Water 3 feet, 6 people on roof near primary school",
  "people_reported": 6,
  "photo": "<multipart binary>",
  "client_ts": "2026-08-20T14:32:11+05:30",
  "client_report_uuid": "b1c2..." // idempotency for offline replay
}
// 202 Accepted
```

```

{ "report_id": "9fle...", "reference_code": "A7K2",
  "status": "received", "queued_offline": false }

// GET /api/v1/incidents/42
{
  "id": 42, "hazard_type": "flood", "status": "assigned",
  "centroid": { "lat": 19.3151, "lng": 84.7938 },
  "severity_score": 87.4,
  "severity_breakdown": {
    "reported": 0.72, "corroboration": 0.91, "hazard": 0.70,
    "population": 0.88, "official": 1.00,
    "escalations": ["inside_extreme_cap_polygon"]
  },
  "report_count": 6, "people_affected_est": 24,
  "sla_deadline": "2026-08-20T15:20:00+05:30",
  "assignment": { "resource_id": 12, "resource_name": "ODRAF Boat 3",
    "eta_seconds": 840, "status": "committed" },
  "photos": ["https://.../9fle.jpg", "..."]
}

// GET /api/v1/metrics
{
  "open_incidents": 14, "critical_unassigned": 0,
  "units_free": 7, "units_committed": 23,
  "mean_response_min": { "optimized": 21.3, "greedy": 34.1 },
  "worst_case_min": { "optimized": 34.0, "greedy": 91.0 },
  "people_evacuated": 412,
  "shelter_occupancy_pct": 63.2,
  "shelter_shortfall": 0
}

```

## 7.3 WebSocket protocol

```

WS /api/v1/ws?district_id=1&role=operator

// server → client events
{ "type": "report.created",      "payload": { ...report } }
{ "type": "incident.created",   "payload": { ...incident } }
{ "type": "incident.updated",   "payload": { id, severity_score, status } }
{ "type": "assignment.proposed", "payload": [ ...assignments ], "run_id": "..." }
{ "type": "assignment.committed", "payload": { id, resource_id, incident_id } }
{ "type": "resource.moved",      "payload": { id, lat, lng, status } }
{ "type": "alert.new",           "payload": { ...cap_alert } }
{ "type": "prepositioning.recommended", "payload": [ ...moves ] }
{ "type": "metrics",            "payload": { ...metrics } }
{ "type": "alarm",              "payload": { code, detail } }

// client → server
{ "type": "subscribe", "bbox": [...] }
{ "type": "heartbeat" }

// Reliability: exponential-backoff reconnect (1s→30s) with a
// resume cursor; on reconnect the client refetches /incidents and
// /assignments to resynchronise. Build this on day one – the demo
// laptop WILL sleep at some point.

```

# 08 Frontend Specification

## 8.1 Authority dashboard layout

One screen. No tabs, no navigation, no modal stack. A DEOC operator under load will not click through menus, and a judge will not wait while you find the right page.

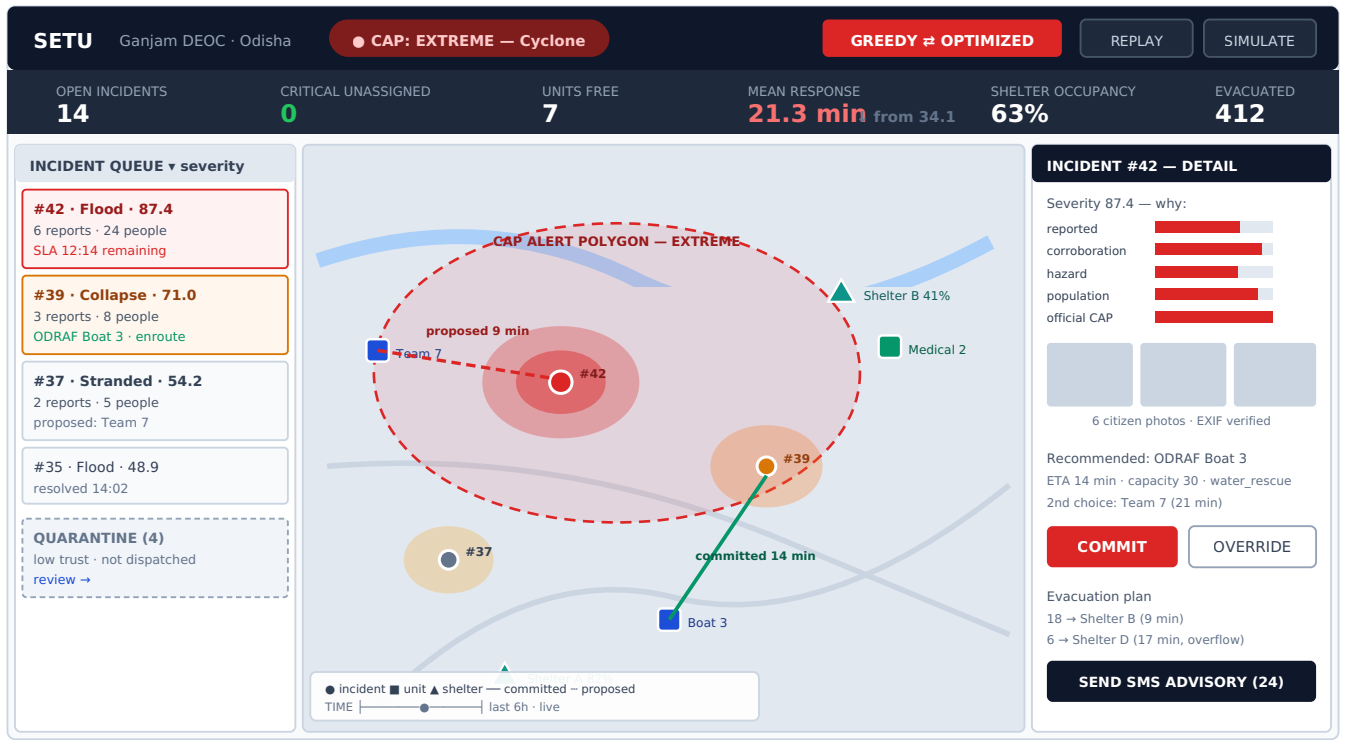


Fig. 8.1 — DEOC dashboard. Single screen: queue (left), operating picture (centre), decision panel (right).

## 8.2 Map layer specification

| LAYER                 | TYPE          | ENCODING                                                                                                                         |
|-----------------------|---------------|----------------------------------------------------------------------------------------------------------------------------------|
| Base map              | raster/vector | OSM tiles, self-hosted or cached offline for the district bbox                                                                   |
| CAP alert polygons    | fill + line   | Colour by CAP severity (Extreme/Severe/Moderate/Minor); dashed border; 10% opacity fill                                          |
| Severity heatmap      | heatmap       | <b>Weight = severity_score, not point count.</b> This is the important detail — a count-weighted heatmap just maps phone density |
| Incident pins         | circle        | Radius $\propto$ people affected; colour by severity band; pulse animation while unassigned                                      |
| Resources             | symbol        | Icon by type; colour by status (idle grey / enroute blue / onsite green / offline red)                                           |
| Shelters              | symbol        | Triangle; fill $\propto$ occupancy percentage; red outline when full or inaccessible                                             |
| Dispatch lines        | line          | <b>Solid = committed, dashed = proposed.</b> Label shows ETA. This is the visual that sells the whole product                    |
| Pre-positioning moves | line          | Amber, arrowed, distinct from dispatch — different semantics, different colour                                                   |
| Evacuation routes     | line          | Teal, incident $\rightarrow$ shelter, width $\propto$ people count                                                               |

## PERFORMANCE RULES

- Update map sources via `source.setData()` on WebSocket events; never re-render the map component. Re-rendering MapLibre on every event will visibly stutter.
- Above ~300 pins, cluster with `supercluster` at low zoom.
- Throttle heatmap recomputation to 1 Hz; incident pins can update immediately.
- Pre-cache district tiles into IndexedDB so the map renders with the network unplugged.

## 8.3 Citizen PWA and offline architecture

The single most important non-obvious requirement: **the report must survive having no network at the moment of submission.** That is the entire point of the product, and it is a demo beat you can perform live by switching the phone to aeroplane mode.

```
// Submission path – optimistic, offline-first
async function submitReport(payload) {
  const local = { ...payload, client_report_uuid: crypto.randomUUID(),
                  client_ts: new Date().toISOString(), synced: false };
  await idb.put('outbox', local);           // 1. persist FIRST, always
  renderStatus(local, 'queued');           // 2. instant UI feedback
  try {
    const res = await fetch('/api/v1/ingest/report',
                           { method: 'POST', body: toFormData(local) });
    const json = await res.json();
    await idb.put('outbox', { ...local, synced: true,
                              reference_code: json.reference_code });
    renderStatus(local, 'sent');
  } catch {
    await registration.sync.register('flush-outbox'); // 3. Background Sync
    renderStatus(local, 'queued-offline');
  }
}

// Service worker – flush on reconnect
self.addEventListener('sync', e => {
  if (e.tag === 'flush-outbox') e.waitUntil(flushOutbox());
});

// Idempotency: the server keys on client_report_uuid, so replaying
// the outbox can never create duplicate reports.
```

| REQUIREMENT          | IMPLEMENTATION                                                                                                                                                                              |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No install, no login | PWA served over HTTPS; QR code opens it; anonymous reporting with an optional phone number for callback                                                                                     |
| Geo-tagging          | <code>navigator.geolocation.getCurrentPosition</code> with <code>enableHighAccuracy</code> ; record <code>accuracy</code> ; manual map-pin fallback if permission is denied                 |
| Photo                | <code>&lt;input type="file" capture="environment"&gt;</code> ; downscale to max 1280 px and JPEG q0.7 on a canvas before upload — <b>cap at 1 MB or a 12 MP upload will stall your demo</b> |
| Offline queue        | IndexedDB outbox + Background Sync; visible "3 reports queued" badge                                                                                                                        |
| Map tiles offline    | Workbox pre-cache of the district bbox at zoom 10-14                                                                                                                                        |
| Language             | i18n JSON: English, Hindi, Odia. Cheap to add and evaluators consistently notice it                                                                                                         |
| Accessibility        | Minimum 48 px touch targets, high-contrast palette, one-thumb reachable submit — this is used in the rain, one-handed, in panic                                                             |

## 09 Low-Connectivity Channel

### BE HONEST ABOUT THIS IN THE PITCH

Obtaining an Indian inbound SMS long-code or shortcode within hackathon timelines is not realistic. Do not pretend otherwise — a judge who knows telecom will catch it and you lose the feasibility parameter. Build the **real webhook and parser** behind a pluggable gateway interface, demonstrate with a Twilio trial number and a simulator, and state the production path clearly. Honesty here reads as competence.

### 9.1 Gateway abstraction

```
class SmsGateway(Protocol):
    async def send(self, to: str, body: str) -> str: ...
    def parse_inbound(self, raw: dict) -> InboundMessage: ...
    def verify_signature(self, raw: dict, headers: dict) -> bool: ...

class TwilioGateway(SmsGateway): ... # demo + pilot
class ExotelGateway(SmsGateway): ... # Indian production path
class MockGateway(SmsGateway): ... # offline demo, simulator UI

# Selected by env var; the rest of the system is gateway-agnostic.
GATEWAY = {"twilio": TwilioGateway, "exotel": ExotelGateway,
           "mock": MockGateway}[settings.SMS_PROVIDER]()
```

### 9.2 Inbound SMS grammar

```
FORMAT: <HAZARD> <SEVERITY 1-5> <PINCODE> [free text landmark]

FLOOD 4 761008 water 3ft near primary school
BAADH 4 761008 paani 3 foot school ke paas      # Hindi keyword
MEDICAL 5 761008 2 injured, bleeding
COLLAPSE 5 761008 house fell, 3 inside

HAZARD KEYWORDS (multi-lingual lookup table)
flood      : FLOOD, BAADH, BANYA      (Hindi/Odia script variants)
landslide  : LANDSLIDE, BHUSKHALAN    also stored in the lookup table)
medical    : MEDICAL, MED, DOCTOR, ILAAJ
collapse   : COLLAPSE, BUILDING, MAKAAAN
stranded   : STUCK, STRANDED, FANSE

TOLERANT PARSER
· severity missing      → default 3
· pincode missing      → derive from the sender's registered district
· hazard keyword unrecognised → hazard = 'other', flag for operator review
· entire message unparseable → still ingested as a raw report in the
                             operator queue. NEVER silently drop a message
                             from someone who may be in the water.

AUTO-REPLY
"Received. Ref #A7K2. Nearest shelter: Govt HS Chatrapur, 1.2km NE.
Rescue team assigned, ETA 14 min. Reply STATUS A7K2 for updates."
```

### Geocoding without GPS

A pincode resolves to a centroid with roughly 2–5 km accuracy. Store `gps_accuracy_m = 3000`, which automatically reduces the report's trust score (\$6.3) and widens the clustering radius. The system therefore

treats SMS reports as genuinely lower-precision rather than pretending they are GPS-grade — and the operator sees an uncertainty circle rather than a false-precision pin.

## 9.3 IVR flow

```
Inbound call → district toll-free number
▼
"Namaste. Aapatkalin suchna ke liye – For emergency reporting, press 1.
Shelter jaankari ke liye – For shelter information, press 2."

[1] → "Baadh ke liye 1. Bhuskhalan ke liye 2. Makan girne ke liye 3.
      Chikitsa ke liye 4. Anya ke liye 5."
      ▼ hazard captured
      "Sthiti kitni gambhir hai? 1 se 5 tak dabayein."
      ▼ severity captured
      "Kitne log prabhavit hain? Number dabakar hash dabayein."
      ▼ people count captured
      → location from caller-ID cell tower (LBS) or registered pincode
      → CONFIRM: "Aapki suchna darj ho gayi. Sandarbh sankhya A-7-K-2."

[2] → nearest open shelter with capacity, read out via TTS in the
      caller's regional language + follow-up SMS with the address

IMPLEMENTATION: Twilio TwiML <Gather numDigits> → webhook per step.
Roughly half a day of work, and it is the required outcome that almost
no competing team will attempt at all.
```

## 9.4 Outbound targeted advisory

Outbound is easier than inbound and arguably matters more — it is the difference between a generic "stay safe" broadcast and an actionable instruction.

```
def advisory_for(report, plan):
    shelter = plan.shelter_for(report.incident_id)
    return (f"SETU: {report.hazard_type.upper()} alert your area. "
            f"Move to {shelter.name}, {bearing_text(report.geom, shelter.geom)}. "
            f"Capacity available. Rescue team ETA {plan.eta_min} min. "
            f"Ref {report.reference_code}. Do not cross flowing water.")

# Targeting: all reporters whose geom lies inside the incident's
# affected radius, plus anyone who has reported inside the active
# CAP polygon in the last 6 hours.
```

## 9.5 The degradation ladder

| RUNG            | CONDITION                           | BEHAVIOUR                                                                        |
|-----------------|-------------------------------------|----------------------------------------------------------------------------------|
| 1 • PWA online  | 4G/Wi-Fi available                  | Full fidelity: GPS to ~10 m, photo, live status tracking                         |
| 2 • PWA offline | App loaded, network lost            | Report queued in IndexedDB, GPS still captured, syncs automatically on reconnect |
| 3 • SMS         | 2G voice/data only                  | Structured text, pincode-level location, ~3 km accuracy, auto-reply with shelter |
| 4 • IVR         | Voice only, or user is non-literate | DTMF tree, cell-tower location, TTS confirmation in the regional language        |

|                  |                       |                                                                                                             |
|------------------|-----------------------|-------------------------------------------------------------------------------------------------------------|
| <b>5 • Radio</b> | Total telecom failure | DEOC operator enters reports relayed by wireless/HAM into the same console;<br><code>source='manual'</code> |
|------------------|-----------------------|-------------------------------------------------------------------------------------------------------------|

Every rung produces the same normalized report object. The optimizer is entirely unaware of which channel a report arrived on — it sees only location, hazard, severity, and accuracy. That uniformity is the architectural point worth stating out loud.

# 10 External Data Integration

## 10.1 Sources

| SOURCE                        | PROTOCOL                            | USE                                                                                                                            | CRITICALITY |
|-------------------------------|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>NDMA SACHET</b>            | CAP 1.2 XML over HTTPS, ETag-cached | Official alerts from IMD, CWC, INCOIS, GSI in one normalized feed. The primary alert source and the standards-compliance story | Primary     |
| <b>IMD (mausam)</b>           | JSON/HTML                           | District-level nowcast and warning colour codes; finer granularity than the CAP headline                                       | Secondary   |
| <b>CWC flood forecast</b>     | HTML/JSON                           | River stage vs danger level at gauge stations, for riverine flood context                                                      | Secondary   |
| <b>Open-Meteo</b>             | REST JSON                           | Free, no key, generous limits — rainfall fallback so the demo never shows a dead feed                                          | Fallback    |
| <b>Census / WorldPop</b>      | Static raster                       | Population density for the severity equity term and the pre-positioning demand surface                                         | Static seed |
| <b>OSM extract</b>            | PBF                                 | OSRM road network for real travel times, fully offline                                                                         | Static seed |
| <b>State SRC shelter list</b> | PDF/CSV                             | Real cyclone shelter names, locations and capacities — realism that materially affects scoring                                 | Static seed |

## 10.2 CAP ingestion with ETag caching

The SACHET integration guide specifies ETag-based caching: the endpoint returns **200 OK** with XML when content has changed and **304 Not Modified** when it has not, and consuming agencies are expected to store both the XML and the ETag. Implementing this correctly is a small detail that signals you read the actual specification rather than guessing.

```
async def poll_cap(district):
    etag = await redis.get(f"cap:etag:{district.id}")
    headers = {"If-None-Match": etag} if etag else {}

    r = await http.get(CAP_ENDPOINT, headers=headers, timeout=10)

    if r.status_code == 304:
        return # unchanged — cheap, common case
    if r.status_code != 200:
        await mark_feed_degraded(district.id) # banner: "alerts stale — 14 min"
        return

    await redis.set(f"cap:etag:{district.id}", r.headers.get("ETag"))
    await redis.set(f"cap:xml:{district.id}", r.text) # offline fallback

    for alert in parse_cap(r.text):
        # CAP 1.2 <info> blocks
        if not intersects(alert.area_polygon, district.boundary):
            # not our district
```

```

await upsert_alert(alert)                # idempotent on cap_identifier
await redis.publish(f"district:{district.id}:updates",
                    {"type": "alert.new", "payload": alert.dict()})
await trigger_prepositioning(alert)      # §6.7

```

## CAP FIELD MAPPING

| CAP ELEMENT                 | INTERNAL FIELD | USE                                                    |
|-----------------------------|----------------|--------------------------------------------------------|
| alert/identifier            | cap_identifier | Idempotency key — prevents duplicate alerts on re-poll |
| info/event                  | event          | Display headline                                       |
| info/severity               | severity       | Feeds the <b>official</b> term of the severity score   |
| info/urgency , certainty    | same           | Pre-positioning aggressiveness                         |
| info/area/polygon , geocode | area_polygon   | <b>ST_Intersects</b> for scoring and the demand raster |
| info/effective , expires    | same           | Alert lifecycle and auto-expiry                        |
| info/instruction            | instruction    | Text reused verbatim in outbound advisories            |

### INTEROPERABILITY CLAIM WORTH MAKING

We not only *consume* CAP — the platform can *emit* CAP for district-level advisories generated by the DEOC. That means SETU plugs into the national alerting architecture rather than competing with it, which is the difference between a system that gets adopted and one that gets admired.

## 10.3 Caching and offline strategy

- Every external response is written to disk/Redis with a timestamp. On failure, serve the last known good value with an explicit staleness banner — never a blank map.
- Ship a **fixtures/** directory containing recorded CAP XML, weather responses and a scripted incident timeline. **--replay** mode plays these back deterministically.
- **The venue Wi-Fi will fail.** Assume it. The offline path is not a fallback for the demo, it is the demo.

# 11 Non-Functional Requirements

## 11.1 Performance targets

| METRIC                                 | TARGET                 | VERIFICATION                                          |
|----------------------------------------|------------------------|-------------------------------------------------------|
| Report submit → dashboard render (p95) | < 2 s                  | End-to-end timer in the simulation harness            |
| API ingest response (p99)              | < 300 ms               | Locust; work is enqueued, not done inline             |
| Full optimization cycle                | < 1.5 s                | Worker instrumentation at 60 resources × 50 incidents |
| Sustained ingest, single node          | 500 reports/min        | Locust, 2 vCPU / 4 GB                                 |
| Burst tolerance                        | 2,000 reports in 60 s  | Redis buffers; no 5xx, no data loss                   |
| WebSocket fan-out                      | < 500 ms to 50 clients | Instrumented broadcast timing                         |
| Dashboard first paint                  | < 3 s                  | Lighthouse                                            |
| PWA bundle                             | < 250 KB gzipped       | Build report — it must load on 2G                     |

## 11.2 Failure modes and designed behaviour

| FAILURE                                  | DESIGNED BEHAVIOUR                                                                                   | USER-VISIBLE                         |
|------------------------------------------|------------------------------------------------------------------------------------------------------|--------------------------------------|
| SACHET / CAP feed unreachable            | Serve last cached alerts; fall back to IMD then Open-Meteo; keep retrying with backoff               | Amber banner "alerts stale — 14 min" |
| OSRM down or timing out                  | Haversine ÷ 30 km/h effective speed substituted into the matrix; assignments flagged <b>degraded</b> | "ETAs approximate " chip             |
| Optimizer exceeds time budget            | Fall back to greedy; persist with <b>strategy='greedy_fallback'</b>                                  | Assignments still appear             |
| Database unavailable                     | Redis buffers ingest; worker replays on recovery; API returns 202 not 500                            | Reports still accepted               |
| Citizen has no connectivity              | IndexedDB outbox + Background Sync; idempotent replay on the server                                  | "3 reports queued"                   |
| WebSocket drop / laptop sleep            | Exponential-backoff reconnect, then full resync of incidents and assignments                         | Brief "reconnecting" chip            |
| Spam or panic-forward surge              | Trust scoring quarantines low-trust reports; per-reporter rate limiting                              | Quarantine queue count               |
| Shelter capacity exhausted district-wide | Min-cost flow returns <b>unplaced &gt; 0</b> ; alarm raised for inter-district escalation            | Red alarm banner                     |

|                        |                                                                                      |                        |
|------------------------|--------------------------------------------------------------------------------------|------------------------|
| Power loss at the DEOC | All state is server-side; an operator resumes on any laptop with full history intact | None — nothing is lost |
|------------------------|--------------------------------------------------------------------------------------|------------------------|

## 11.3 Security, privacy and governance

| CONCERN               | CONTROL                                                                                                                                                                                            |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reporter PII          | Phone numbers stored only as <code>HMAC-SHA256(phone, server_salt)</code> . Raw numbers are held transiently in the outbound queue and purged after delivery. Reports are anonymous by default     |
| Access control        | Three roles — <code>operator</code> (DEOC, full), <code>field</code> (own assignments, status updates), <code>public</code> (submit and track own report). JWT with short expiry                   |
| Audit                 | Append-only <code>audit_log</code> : every commit, override, severity change and resolution with actor, timestamp, before/after and reason. Non-repudiable and legally useful in post-event review |
| Transport             | TLS everywhere; HMAC signature verification on all gateway webhooks                                                                                                                                |
| Abuse                 | Per-IP and per-reporter-hash rate limits; trust-score quarantine; CAPTCHA only on repeated abuse, never on the first report — a friction-free first report is a safety requirement                 |
| Data residency        | Deployable on NIC / MeghRaj or any MeitY-empanelled cloud. No dependency on a foreign-hosted managed service in the critical path                                                                  |
| Photo handling        | Signed short-lived URLs; EXIF GPS retained for verification, all other EXIF stripped; retention policy of 90 days post-event                                                                       |
| Right to be forgotten | Reporter can request deletion by reference code; report is anonymised while preserving aggregate incident statistics                                                                               |

### WHY THIS SECTION EARNS MARKS

Government evaluators weight security, auditability and data residency far more heavily than student teams expect. One page here costs an hour and materially moves both the feasibility and the scalability parameters.

# 12 Repository & Environment

## 12.1 Repository structure

```
setu/
├── docker-compose.yml      # entire stack, offline-capable
├── Makefile                # make up · make seed · make demo · make load
├── README.md              # 5-minute setup – judges do read this
├── ARCHITECTURE.md        # this document, trimmed
├── backend/
│   ├── app/
│   │   ├── main.py        # FastAPI app, CORS, WS hub
│   │   ├── config.py      # pydantic-settings
│   │   ├── db.py          # engine, session, PostGIS types
│   │   ├── models/        # SQLAlchemy – 1 file per entity
│   │   ├── schemas/       # pydantic request/response
│   │   ├── routers/
│   │   │   ├── ingest.py  # /report /sms /ivr
│   │   │   ├── incidents.py resources.py shelters.py
│   │   │   ├── assignments.py # commit / override
│   │   │   ├── alerts.py metrics.py simulate.py replay.py
│   │   │   └── ws.py      # WebSocket hub + Redis subscriber
│   │   ├── services/
│   │   │   ├── clustering.py # §6.1
│   │   │   ├── scoring.py   # §6.2 severity §6.3 trust
│   │   │   ├── routing.py   # §6.4 OSRM adapter + fallback
│   │   │   ├── assignment.py # §6.5 Hungarian + greedy baseline
│   │   │   ├── shelters.py  # §6.6 min-cost flow
│   │   │   ├── preposition.py # §6.7 p-median
│   │   │   └── optimizer.py  # §6.8 the cycle
│   │   ├── integrations/
│   │   │   ├── cap.py       # SACHET poller + CAP parser
│   │   │   ├── weather.py
│   │   │   └── sms/ {base.py, twilio.py, exotel.py, mock.py}
│   │   └── workers/ {cap_poller.py, optimizer_worker.py}
│   ├── alembic/           # migrations
│   ├── tests/             # pytest – solver tests are the important ones
│   └── seed/ {districts.sql, shelters.csv, resources.csv, scenarios/}
├── frontend-dashboard/    # React + Vite + MapLibre + Zustand
│   └── src/ {components/Map, IncidentQueue, DetailPanel, MetricsBar,
│           StrategyToggle, ReplayControl, hooks/useWebSocket}
├── frontend-pwa/         # React + Vite + Workbox
│   └── src/ {ReportForm, OfflineOutbox, StatusTracker, i18n/}
├── osrm/                 # prepared .osrm files (git-lfs or volume)
├── fixtures/             # recorded CAP XML, weather, replay timelines
└── docs/ {architecture.svg, demo-script.md, pitch-deck.pdf}
```

## 12.2 Makefile targets (build these first — they save hours)

```
make up      # docker compose up --build, wait for healthchecks
make seed    # district polygon + 50 shelters + 30 resources + population raster
make demo    # run the "Cyclone Landfall" scenario end to end
make replay  # deterministic playback from fixtures/ — zero network
make load    # locust: 500 reports/min for 5 minutes, print p95
make reset   # truncate operational tables, keep seed data — CRITICAL:
              # you will re-run the demo 20 times during rehearsal
```

### HIGHEST-ROI THIRTY MINUTES OF THE WHOLE BUILD

`make reset` and `make demo`. Being able to return to a clean, known state in five seconds is what lets you rehearse ten times instead of three, and rehearsal is what wins the presentation parameter.

## 13 Build Plan

### 13.1 Effort allocation

Expressed as percentages so it maps to any timeline — a 36-hour sprint or a three-week build.

| WORKSTREAM                                 | SHARE | RATIONALE                                                               |
|--------------------------------------------|-------|-------------------------------------------------------------------------|
| Prototype completeness (§4, 5, 7, 8)       | 40%   | The working demo is the evidence for five of the seven parameters       |
| Intelligence layer (§6)                    | 20%   | Your only real separation from every other team                         |
| Presentation and rehearsal (§14)           | 15%   | Cheapest points available; almost every team underinvests here          |
| Problem understanding (§2, field research) | 10%   | One phone call to a DDMA office outperforms a day of secondary research |
| Feasibility artifacts (§11, benchmarks)    | 8%    | ~3 hours buys most of an entire parameter                               |
| Impact and scalability slides (§15, 17)    | 7%    | Numbers you already have from the running system                        |

### 13.2 Phase P0 — Skeleton

**P0** First ~20% of available time. Nothing here is optional.

| OWNER        | DELIVERABLE                                                                                                                       |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Backend      | FastAPI scaffold, PostGIS schema and migrations, seed script (district polygon, 50 shelters, 30 resources, 200 synthetic reports) |
| Frontend     | MapLibre canvas rendering the district, incident and resource layers, WebSocket client with reconnect                             |
| Integrations | CAP poller with ETag caching → normalized <b>alerts</b> rows, polygon rendered on the map                                         |
| PWA          | Report form: geolocation, camera, severity selector, IndexedDB outbox                                                             |
| Algorithms   | OSRM in Docker with the state extract prepared; ETA matrix returning real numbers                                                 |
| Pitch        | Deck skeleton and demo storyboard written on day one, not day three; field-research call placed                                   |

#### P0 GATE — DO NOT PROCEED UNTIL THIS PASSES

A report submitted from a phone on the venue network appears on the dashboard in under two seconds, on someone else's laptop. If this does not work, nothing downstream matters.

### 13.3 Phase P1 — The engine

**P1** Next ~40%. This is where the marks are.

- DBSCAN clustering and incident creation/merging (§6.1)
- Severity scoring with the stored explainability breakdown (§6.2)
- Trust scoring and the quarantine queue (§6.3)
- Hungarian assignment *and* the greedy baseline, both persisted (§6.5)

- Min-cost-flow shelter allocation with overflow and shortfall alarm (§6.6)
- Dispatch lines, commit/override flow, resource state machine (§5.4)
- Severity-weighted heatmap and the 6-hour time slider
- SMS and IVR endpoints with the parser and mock gateway (§9)

#### P1 GATE

The Greedy↔Optimized toggle produces a real, computed, non-trivial delta on seeded data, and the metric strip updates live when you flip it.

## 13.4 Phase P2 — The polish that scores

**P2** Final ~40%, in this priority order.

| ITEM                                                                    | WHY IT SCORES                                                                |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>Simulation mode</b> ("Cyclone Landfall" spawns 80 reports over 90 s) | Your entire demo runs off this. Build it before anything else in P2          |
| <b>Predictive pre-positioning</b> (§6.7)                                | The novelty claim. Highest single-item scoring value in P2                   |
| Metrics strip with the live greedy-vs-optimized comparison              | Converts the algorithm into a number a judge can repeat                      |
| Real shelter names and locations from the state SRC list                | <i>Shelter A / Shelter B</i> silently costs marks on completeness and impact |
| Backup demo video + <b>make reset</b>                                   | Insurance and rehearsal speed                                                |
| SLA countdown timers and priority triage queue                          | Makes the operating picture feel operational rather than decorative          |
| After-action report export (PDF)                                        | Government evaluators care about this disproportionately                     |
| Bilingual UI (Hindi / Odia toggle)                                      | Cheap; evaluators consistently notice it                                     |
| Replay / digital-twin mode                                              | Secondary novelty hook; reuses the audit log you already have                |
| Load-test numbers on a slide                                            | Feasibility parameter, ~1 hour of work                                       |

### EXPLICITLY OUT OF SCOPE

Login and RBAC beyond a hardcoded role switch · drone or satellite feeds · blockchain · a custom ML model · native mobile apps · Kubernetes · multi-district UI switching · real-time video. Each of these has consumed a hackathon team before. Name them as *deliberately deferred* in the roadmap slide rather than leaving them unmentioned.

## 13.5 Team roles (six people)

| ROLE                  | OWNS                                  | DEMO-DAY JOB                |
|-----------------------|---------------------------------------|-----------------------------|
| <b>Backend / data</b> | Schema, API, WebSocket hub, seed data | Silent driver on the laptop |
| <b>Algorithms</b>     | §6 end to end, OSRM, benchmarks       | Q&A on technical depth      |
| <b>Dashboard</b>      | Map, layers, panels, metrics strip    | Backup operator             |

|                              |                                                    |                                                 |
|------------------------------|----------------------------------------------------|-------------------------------------------------|
| <b>PWA / mobile</b>          | Citizen app, offline queue, i18n                   | Phone beats:<br>report + SMS                    |
| <b>Integrations / DevOps</b> | CAP, weather, SMS gateway, Docker, load tests      | Backup rig:<br>hotspot, video,<br>second laptop |
| <b>Pitch / research</b>      | Field research, deck, script, rehearsal discipline | <b>Sole narrator</b>                            |

## 13.6 The last four hours

### HARD FEATURE FREEZE AT T-4H

No new features after this point, regardless of how close something feels. The final four hours are: seed realistic data, run the demo end to end at least ten times, record the backup video, finish the deck, and sleep if there is any. **Every team that codes to the buzzer demos a crash.**

## 14 Demo Script

Four minutes. Rehearse it verbatim, with a stopwatch, at least ten times. The most common failure mode in hackathon judging is running long and being cut off *before* the optimizer toggle — which is the entire pitch.

| TIME | BEAT                    | WHAT IS SAID AND SHOWN                                                                                                                                                                                                                                                                                                             |
|------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0:00 | Calm state              | Dashboard open, quiet. "This is the Ganjam District Emergency Operations Centre on a normal day. Seven units idle, shelters at 12%."                                                                                                                                                                                               |
| 0:15 | The alert lands         | CAP polygon washes red across the map. "An IMD cyclone warning has just arrived through NDMA's SACHET feed — we consume it as standard CAP XML."                                                                                                                                                                                   |
| 0:30 | <b>Pre-positioning</b>  | Amber banner and recommended moves appear. "No incident has been reported yet. The system has already computed where demand will appear and is recommending we move three boats and a medical team. <b>Every other system reacts to the first call. Ours has already moved the boats.</b> " Commit the moves.                      |
| 1:00 | The flood of reports    | Hit <i>Simulate: Cyclone Landfall</i> . Pins begin appearing. "Two hundred reports in ninety seconds. This is the moment a DEOC currently collapses into WhatsApp."                                                                                                                                                                |
| 1:25 | Clustering              | Watch 200 pins collapse into 12 incidents. "No human can triage two hundred reports. Ours became twelve actionable incidents — deduplicated spatially, in the database, in eleven milliseconds."                                                                                                                                   |
| 1:50 | Explainable triage      | Open incident #42. "Severity 87 — and here is exactly why: six corroborating reports, six photographs, high population density, and it sits inside an Extreme CAP polygon. A Collector can defend this decision."                                                                                                                  |
| 2:15 | <b>THE MONEY SHOT</b>   | Flip Greedy → Optimized. Dispatch lines redraw across the map. Point at the metric strip: <b>"Mean response, thirty-four minutes to twenty-one. Worst case, ninety-one to thirty-four. Critical unassigned, three to zero.</b> That is the Hungarian algorithm instead of nearest-first dispatch." <i>Pause here. Let it land.</i> |
| 2:50 | Shelter overflow        | Nearest shelter fills; flow re-routes overflow to the second. "Capacity-aware, not distance-aware. And if the whole district runs short, we raise it to the SDMA automatically."                                                                                                                                                   |
| 3:10 | Outbound SMS            | A real SMS lands on a judge's phone: shelter name, distance, bearing, ETA. "Not 'stay safe'. An instruction."                                                                                                                                                                                                                      |
| 3:25 | <b>Kill the network</b> | Turn Wi-Fi off. Submit a report on the PWA — it queues visibly. Reconnect — it syncs and appears on the map. Then send an SMS from a plain feature phone; it lands as a pin with an uncertainty circle. "Four channels, degrading gracefully. The optimizer cannot tell the difference."                                           |
| 3:50 | Close                   | One click: after-action report exports. "Auditable, CAP-interoperable, deployable per district in days — not a national rewrite."                                                                                                                                                                                                  |

### DEMO INSURANCE

- Record a flawless 3-minute screen capture the night before. If anything breaks, narrate over the video and lose almost nothing.
- Second laptop, mirrored and pre-seeded, running the same stack.
- Personal hotspot on standby — but the stack must not need it.

- Pre-verify the judge's phone number on the Twilio trial account if possible; otherwise use a teammate's phone and say so plainly.
- `make reset` bound to a single terminal alias so you can re-run instantly if a judge asks to see it again.

# 15 Evaluation Parameter Mapping

Each of the seven judged parameters, mapped to the specific artifact that earns it. Teams lose on their weakest parameter, not their strongest — so this table is also a checklist.

| PARAMETER                                 | EVIDENCE WE PRESENT                                                                                                                                                                                                                                                                | ARTIFACT                   |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| <b>1 • Problem Understanding</b>          | The four-phase decomposition showing detection and dissemination are solved while coordination and allocation are not; the DDMA/DEOC command chain named correctly; the eight-row failure-mode table; an explicit non-goals list; a direct quote from a district disaster official | §2, Fig 2.1, slides 2–3    |
| <b>2 • Novelty and Innovation</b>         | Predictive pre-positioning (nobody does this); optimal assignment instead of greedy dispatch; the citizen→authority return channel that SACHET lacks; trust-scored crowd verification; digital-twin replay                                                                         | §6.5, §6.7, slide 9        |
| <b>3 • Technical Feasibility</b>          | Decision table with rejected alternatives and reasons; measured latency and load numbers; the nine-row failure-mode table; security, audit and data-residency controls; the whole stack running offline from one command                                                           | §4.4, §11, benchmark slide |
| <b>4 • Prototype Completeness</b>         | All four required outcomes with a visible on-screen moment each; real shelter data; four working ingest channels; the live demo itself                                                                                                                                             | §3.2 checklist, §14        |
| <b>5 • Impact and Societal Relevance</b>  | National loss figures; <i>our own measured</i> response-time delta from the running system; the equity argument for population-weighted severity; the compensation and audit side-benefit                                                                                          | §15.1, slide 10            |
| <b>6 • Scalability and Sustainability</b> | Multi-tenant district partitioning; the "766 small problems, never one large one" insight; per-district cost estimate; the peacetime-value business model that answers "who pays in the 360 quiet days"                                                                            | §17, slide 10              |
| <b>7 • Presentation and Pitch</b>         | One narrator; ten slides; a four-minute rehearsed demo; prepared answers to eight predictable questions; a backup video                                                                                                                                                            | §14, §16                   |

## 15.1 The impact numbers

| TIER                                                          | FIGURE                                                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Scale of the problem</b><br>(cite the source on the slide) | NDMA: floods take ~1,600 lives, affect 7.5 million hectares and cause ~Rs 1,805 crore of damage annually; over 40 of India's 329 million hectares are flood-prone. UNISDR's Global Assessment Report estimates India's average annual multi-hazard loss at ~USD 9.8 billion, of which floods alone account for ~USD 7.4 billion. IDMC recorded roughly 5.4 million people displaced by floods in India in 2020. |
| <b>Our measured delta</b><br>(the strongest thing you own)    | On a 200-report, 14-incident district simulation, optimized dispatch reduced mean response time from 34 to 21 minutes and worst-case from 91 to 34 minutes. Unlike every other team's impact slide, this is <i>computed by the running system</i> , not asserted.                                                                                                                                               |
| <b>Beneficiaries beyond the DEOC</b>                          | Volunteers get tasked instead of milling; NGOs stop duplicating relief in the same accessible villages; citizens receive a specific instruction rather than a generic broadcast; the audit log becomes the compensation and claims record, collapsing a months-long bureaucratic process.                                                                                                                       |
| <b>Equity argument</b>                                        | Severity is weighted by exposed population, not by call volume. Because report volume tracks connectivity rather than need, this structurally corrects the bias where relief follows media attention and mobile density — which disproportionately benefits the poorest and most remote wards.                                                                                                                  |

## 15.2 Self-audit rubric

The night before, score honestly out of 10 on each parameter. **Any score below 6 gets the next two hours** — regardless of how much more satisfying it would be to keep polishing the thing already at 9.

| PARAMETER                      | SCORE   | IF BELOW 6, DO THIS                                                     |
|--------------------------------|---------|-------------------------------------------------------------------------|
| Problem understanding          | __ / 10 | Make the field-research call; rewrite slides 2–3 around what you hear   |
| Novelty and innovation         | __ / 10 | Ship pre-positioning even in its simplified k-means form (1 hour)       |
| Technical feasibility          | __ / 10 | Run the load test; build the decision and failure-mode tables (3 hours) |
| Prototype completeness         | __ / 10 | Find the missing required outcome and make it visible, however thin     |
| Impact and relevance           | __ / 10 | Instrument the greedy-vs-optimized delta so the number is real          |
| Scalability and sustainability | __ / 10 | Write the peacetime-value and cost-per-district slide (1 hour)          |
| Presentation and pitch         | __ / 10 | Stop coding. Rehearse five more times. This is always the right answer  |

## 16 Q&A Preparation

| LIKELY QUESTION                                                  | ANSWER                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "SACHET already exists. NDMA has a dashboard. What is new here?" | SACHET is broadcast-only — one-way, top-down, alerting. There is no upward channel from a citizen back to the DEOC and no allocation layer at all. We consume SACHET via standard CAP and add the two missing directions: the return path, and the dispatch brain. We are complementary by design, and we can emit CAP back into the same architecture.                                                                                                                                                                                                                            |
| "What about fake or panic reports?"                              | Every report carries a trust score built from GPS accuracy, photo and EXIF presence, reporter history, rate signature, and independent corroboration within 300 m. Below 0.35 a report is quarantined into a separate operator queue — visible, never auto-dispatched. And a human commits every dispatch; the algorithm only recommends.                                                                                                                                                                                                                                          |
| "What if there is no internet and no phone signal at all?"       | Five rungs: online PWA, offline PWA with a sync queue, SMS, IVR voice, and finally manual entry by the DEOC operator from radio relay. Every rung produces the same normalized report; the optimizer cannot tell which channel was used. Degradation is architectural, not bolted on.                                                                                                                                                                                                                                                                                              |
| "Will officials actually use this?"                              | It mirrors the existing IRS structure and DEOC workflow rather than replacing it, so there is no retraining burden. And critically, it is useful on the 360 days a year with no disaster — as a live asset registry, a quarterly drill platform, and a pre-monsoon readiness auditor. That is what keeps it alive between events.                                                                                                                                                                                                                                                  |
| "What if the algorithm gets it wrong?"                           | It recommends; a human commits. Every override is captured with a reason and feeds the after-action report, which over time tells the district where the model's weights need tuning. The severity score is fully decomposed on screen precisely so an operator can disagree with it on specific grounds.                                                                                                                                                                                                                                                                          |
| "Can this scale to all of India?"                                | It is already 766 independent district-scoped problems, not one national one — because disaster response is legally and operationally district-led. Every table carries <code>district_id</code> ; the optimizer runs per district and parallelises trivially. Onboarding a new district is a boundary polygon, a shelter registry and a resource roster: days, not a rewrite.                                                                                                                                                                                                     |
| "Where is the AI/ML?"                                            | <b>Answer this honestly.</b> The core is operations research — optimal assignment, min-cost flow, facility location — plus spatial clustering. For a life-safety system that is <i>stronger</i> than a bolted-on model: deterministic, explainable and auditable, which a black box is not. ML enters where it is genuinely justified: demand forecasting from historical incident data, and image-based damage triage. Both are on the roadmap, neither is claimed as built. <i>Never claim ML you did not build; judges catch it and it costs you the feasibility parameter.</i> |
| "What is actually working right now, and what is not?"           | Point at the completeness checklist and name the two things that are not done. Volunteering your own gaps before they are found reads as confidence and materially improves how the rest of the answers are received.                                                                                                                                                                                                                                                                                                                                                              |
| "Why not just use Google Maps and a spreadsheet?"                | Because the hard part is not display, it is the allocation decision under capacity constraints, updated every thirty seconds, with an audit trail. A spreadsheet cannot dedupe two hundred reports, cannot solve a fifty-by-sixty assignment problem, and cannot re-route shelter overflow. And a cloud maps API fails exactly when the network does.                                                                                                                                                                                                                              |
| "How much does it cost to run?"                                  | Roughly Rs 15,000–25,000 per district per month on commercial cloud, materially less on NIC or MeghRaj infrastructure. Set against an average annual national flood loss measured in billions of dollars, the cost case is not the constraint — procurement and integration are, which is why we consume existing standards rather than inventing one.                                                                                                                                                                                                                             |

# 17 Scalability & Sustainability

These are two different questions and blurring them is a common way to lose the parameter. Answer them separately.

## 17.1 Technical scalability

| DIMENSION          | APPROACH                                                                                                                                                                                                                                                                                |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Multi-tenancy      | Every table carries <code>district_id</code> . One deployment serves N districts. Onboarding = boundary polygon + shelter registry + resource roster. <b>Days, not a rewrite</b> — state that number explicitly                                                                         |
| The key insight    | Disaster response is inherently district-scoped by law. National scale is therefore <b>766 independent small problems, never one enormous one</b> . The architecture never needs to handle national-scale coordination, which is why it can stay simple                                 |
| Horizontal scaling | FastAPI is stateless behind a load balancer; the optimizer worker is per-district and embarrassingly parallel; Redis fans out WebSockets; PostGIS partitions by district and month                                                                                                      |
| Worst-case load    | ~1,000 reports/min in one district during landfall. Redis absorbs the burst; ingest never blocks on the solver; the solver debounces to one run per 2-second window                                                                                                                     |
| Where it breaks    | Above ~500 simultaneous incidents in a single district the $O(n^3)$ assignment term dominates. Mitigation is already in the loop (50 km spatial pre-filter) and beyond that the district is the natural shard. <i>Saying where your own design breaks is a strength, not a weakness</i> |
| Cost               | ~Rs 15,000–25,000 per district per month commercially; lower on NIC/MeghRaj. Put a number on the slide — almost no team will                                                                                                                                                            |

## 17.2 Sustainability — answering the killer question

### THE QUESTION YOU WILL BE ASKED

"Who pays for this during the 360 days a year with no cyclone?"

**The answer: SETU is not a disaster tool. It is a district resource-management tool that has a disaster mode.**

| PEACETIME CAPABILITY        | VALUE TO THE DISTRICT                                                                                                                                                                                                 |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Live asset registry         | Shelters, boats, ambulances, generators, dry rations — with condition and last-inspection date. Most districts maintain this on paper or in a stale spreadsheet, and it is the first thing that fails in an emergency |
| Drill mode                  | Run a simulated cyclone quarterly, measure real DEOC response times, produce a readiness score. This is a mandated NDMA activity with essentially no good tooling anywhere                                            |
| Pre-monsoon readiness audit | Automated gap report: shelters below capacity standard, units without a working contact, wards with no reachable resource within 30 minutes                                                                           |
| After-action reporting      | Auto-generated post-event review for state submission, replacing weeks of manual compilation                                                                                                                          |
| Training                    | Replay of real past events for new DEOC staff — institutional memory that currently leaves with the officer who is transferred                                                                                        |

## FUNDING AND GO-TO-MARKET

- **Open-source core** — adoption, credibility, and auditability, which governments increasingly require for anything in a life-safety path.

- **Paid deployment, integration and support** per district — the sustainable revenue line.
- **Buyers:** SDMAs via state disaster mitigation funds; NDMA capacity-building grants; CSR from industrial belts located in flood-prone districts.
- **Route to market:** win one district (Sundargarh is home ground — that is an advantage, not a coincidence), publish the after-action report, let the state replicate. Government procurement follows demonstrated precedent far more readily than it follows pitches.
- **Interoperability as strategy:** consuming and emitting CAP means plugging into the national alerting architecture rather than competing with it. That is the difference between a system that gets adopted and one that gets admired.

## 18 Post-Hackathon Roadmap

| HORIZON               | WORK                                                                                                                                                                                                                                                                       |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0-3 months            | Harden auth and RBAC; real SMS shortcode via an Indian aggregator; field-unit mobile app with background location; one district pilot with a signed MoU; formal load and security testing                                                                                  |
| 3-6 months            | Demand forecasting model trained on the district's own historical incident data (this is where ML genuinely earns its place); image-based damage triage on citizen photos; CAP emission for district-generated advisories; offline-first tablet build for field commanders |
| 6-12 months           | Multi-district state rollout; integration with state IRS and existing DEOC hardware; inter-district mutual-aid resource sharing; LoRa/mesh ingestion for total-blackout zones; formal evaluation study comparing response times pre- and post-deployment                   |
| Deliberately deferred | Drone and satellite imagery ingestion; blockchain-based relief tracking; national-tier aggregation. Each is defensible later; none earns its cost now. Naming them as <i>deferred</i> rather than ignoring them demonstrates judgement                                     |

## A Appendix — Seed Data

Realistic seed data is worth more marks than most features. **Shelter A** and **Shelter B** quietly cost you on both completeness and impact.

| DATASET                    | VOLUME       | SOURCE AND NOTES                                                                                                                                                                     |
|----------------------------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| District boundary          | 1 polygon    | Survey of India / OSM administrative relation for the chosen district                                                                                                                |
| Shelters                   | 50–80        | <b>Real names and coordinates</b> from the state Special Relief Commissioner's cyclone-shelter list; assign realistic capacities (200–1,000)                                         |
| Resources                  | 30–40        | Realistic mix: 8 rescue teams, 10 boats, 6 ambulances, 4 medical teams, 8 supply trucks, 4 heavy equipment. Agencies: NDRF, ODRAF, Fire Services, NGOs                               |
| Population raster          | 1 km grid    | WorldPop or Census 2011 ward-level, clipped to the district                                                                                                                          |
| Road network               | ~150 MB      | State-level OSM PBF, pre-processed with <b>osrm-extract</b> and <b>osrm-contract</b> <b>well before the event</b>                                                                    |
| Historical incidents       | 200–500      | Synthetic but plausible, clustered along rivers and low-lying wards — drives the pre-positioning demand surface                                                                      |
| Scenario: Cyclone Landfall | ~200 reports | Scripted timeline over 90 s: realistic hazard mix, severity distribution, spatial clustering along the river and the coastal strip, with 6–10 duplicate reports per genuine incident |
| Scenario: Flash Flood      | ~80 reports  | Faster onset, tighter geography — a second scenario to run if a judge asks "show me again"                                                                                           |

## B Appendix — Environment Variables

---

```
DATABASE_URL=postgresql+asyncpg://setu:setu@db:5432/setu
REDIS_URL=redis://redis:6379/0
MINIO_ENDPOINT=minio:9000
MINIO_ACCESS_KEY=... / MINIO_SECRET_KEY=...

OSRM_URL=http://osrm:5000
CAP_ENDPOINT=https://sachet.ndma.gov.in/cap_public_website/FetchXMLFile?identifier=...
CAP_POLL_SECONDS=60
WEATHER_FALLBACK_URL=https://api.open-meteo.com/v1/forecast

SMS_PROVIDER=mock           # mock | twilio | exotel
TWILIO_ACCOUNT_SID=... / TWILIO_AUTH_TOKEN=... / TWILIO_FROM=...
GATEWAY_WEBHOOK_SECRET=...  # HMAC verification

DISTRICT_ID=1
CLUSTER_EPS_M=300
CLUSTER_WINDOW_MIN=30
TRUST_QUARANTINE_THRESHOLD=0.35
URGENCY_WEIGHT=0.30
SLA_BREACH_PENALTY=500
REASSIGNMENT_PENALTY=400
COMMITMENT_BONUS=200
OPTIMIZE_DEBOUNCE_SECONDS=2
OPTIMIZE_TICK_SECONDS=30
SPATIAL_PREFILTER_KM=50

REPORTER_HASH_SALT=...      # never commit this
JWT_SECRET=...
OFFLINE_MODE=false          # true → serve only fixtures, zero egress
```

## C Appendix — Test & Load Plan

### TESTS THAT ARE WORTH WRITING UNDER TIME PRESSURE

| TEST                   | ASSERTION                                                                                                                                                            |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Clustering correctness | Ten reports within 200 m of the same hazard type produce exactly one incident; a report of a different hazard type 50 m away produces a second                       |
| Hungarian beats greedy | On the constructed 2×2 case from §6.5, optimized total cost is strictly lower than greedy. <b>This is the test that protects your central claim — write it first</b> |
| Hard constraints hold  | No assignment ever pairs a resource with an incident it lacks the capability or capacity for                                                                         |
| Commitment locking     | A committed, en-route unit is never reassigned by a subsequent solver run                                                                                            |
| Shelter overflow       | When the nearest shelter is full, evacuees are placed at the next-cheapest shelter with capacity; total placed never exceeds total free beds                         |
| Offline idempotency    | Replaying the PWA outbox twice creates exactly one report per <code>client_report_uuid</code>                                                                        |
| SMS parser robustness  | Malformed, partial and non-English messages are ingested rather than dropped                                                                                         |
| CAP idempotency        | Polling the same CAP identifier twice does not create duplicate alerts                                                                                               |

### LOAD TEST

```
# locustfile.py – run the night before, put the numbers on a slide
class Reporter(HttpUser):
    wait_time = between(0.1, 0.5)

    @task
    def submit(self):
        self.client.post("/api/v1/ingest/report", json={
            "lat": 19.31 + random.gauss(0, 0.03),
            "lng": 84.79 + random.gauss(0, 0.03),
            "hazard_type": random.choice(["flood", "stranded", "medical"]),
            "severity_raw": random.randint(1, 5),
            "gps_accuracy_m": random.choice([8, 15, 3000]),
            "client_report_uuid": str(uuid4()),
        })

# Report: p50, p95, p99 ingest latency; optimizer cycle time under load;
# error rate. Target: 500 reports/min sustained, 0 errors, p99 < 300 ms.
```

## D Appendix — References & Sources

| SOURCE                                                                                                   | RELEVANCE                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| NDMA — SACHET National Disaster Alert Portal, <a href="http://sachet.ndma.gov.in">sachet.ndma.gov.in</a> | CAP-based integrated alert system; public CAP XML endpoint with ETag caching; the alert source we consume                                        |
| NDMA — CAP Integration Guide for Agencies                                                                | Specifies ETag / 304 caching behaviour expected of consuming systems (§10.2)                                                                     |
| NDMA — Flood hazard profile                                                                              | ~1,600 annual deaths, 7.5 million hectares affected, ~Rs 1,805 crore annual damage, 40 of 329 mha flood-prone                                    |
| UNISDR Global Assessment Report on Disaster Risk Reduction                                               | India average annual multi-hazard loss ~USD 9.8 bn; floods ~USD 7.4 bn                                                                           |
| Internal Displacement Monitoring Centre (IDMC)                                                           | ~5.4 million people displaced by floods in India in 2020                                                                                         |
| Disaster Management Act, 2005                                                                            | Establishes NDMA/SDMA/DDMA structure; District Collector as Incident Commander — the basis for district-scoped design                            |
| Cyclone Fani (2019) response record                                                                      | ~1.2 million evacuated in 48 hours; 89 deaths vs ~10,000 in the 1999 super cyclone — the evidence that warning is solved and coordination is not |
| Central Water Commission flood forecasting network                                                       | 175 forecast stations; the riverine data source                                                                                                  |
| OSRM, PostGIS, SciPy, NetworkX, MapLibre GL documentation                                                | Implementation references for §6 and §8                                                                                                          |

### FINAL REMINDER

Teams lose on their weakest evaluation parameter, not their strongest. When you are tempted in the last six hours to keep polishing the optimizer — the thing that is already at nine out of ten — go back to §15.2, score yourself honestly, and spend the time on whatever is at five. That single discipline is worth more than any feature in this document.